University of Calabria
Department of Mathematics and Computer Science



Master's Degree Course in Computer Science

Master's Thesis

Lazy Declarative Heuristics in Answer Set Programming: Design and Evaluation of a clingo Propagator

Supervisor at the University of Calabria:
Prof. Carmine Dodaro

Supervisors at the University of Potsdam:
Prof. Dr. Torsten Schaub
Dr. Javier Romero Dávila

Candidate:
Pierpaolo Spadafora

Student ID: 263722

Academic Year 2025/2026

Contents

Abstract **iii**

1 Introduction **1**
1.1 Objective and Contributions 3
1.2 Thesis Organisation 3

2 Background **4**
2.1 Answer Set Programming 4
2.2 Ground-and-Solve: gringo, clasp, and Conflict-Driven Search 5
2.3 Domain-Specific Heuristics in clingo 5
2.4 Lazy Grounding, Alpha, and Heuristics over Partial Assignments . . . 6
2.5 Extending clingo: Propagators and the Heuristic Interface 6

3 Methodology **8**
3.1 The Experimental Design: Two Axes, Four Variants 8
3.1.1 Simulating Alpha semantics 9
3.1.2 Methodological Rationale for the Exploratory Variants 11
3.1.3 Weights, Priorities, and Flattening 12
3.1.4 The Bounded Weight Field 13
3.2 System Architecture 14
3.3 Translation of the Heuristic Directives 16
3.3.1 Lazy heuristic syntax for the C++ backend 16
3.3.2 Lazy heuristic syntax for the Prolog backend 19
3.3.3 What the Two Languages Cannot Express 20

4 Implementation and Benchmark Setup **22**
4.1 Inside the Native Backend 22
4.1.1 Parser and Runtime Representation 22
4.1.2 Incremental Aggregates 22
4.1.3 Evaluation and Decision 23
4.2 The Prolog Backend 24
4.2.1 Rule Strings and Normalization 24
4.2.2 The Runtime Program 25
4.2.3 Synchronization and Decision 25
4.3 Benchmark Problems and Encodings 26
4.3.1 Balanced Sum Problem (BSP) 26
4.3.2 Partner Units Problem (PUP) 27

4.3.3 House Reconfiguration Problem (HRP) . . . 27
4.3.4 Alpha as an External Reference . . . 28
4.4 Benchmark Infrastructure and Metrics . . . 29

5 Results **30**
5.1 Experimental Setup and Dataset . . . 30
5.1.1 Campaign Overview and Failure Modes . . . 31
5.2 Reduction of the Ground Heuristic Component . . . 33
5.3 BSP: Runtime Behaviour . . . 33
5.4 BSP: Rules, Variables, and Memory . . . 38
5.5 PUP: the Grounding Bottleneck at Scale . . . 40
5.6 HRP: Guidance Without Dynamic Aggregates . . . 44
5.7 C++ versus Prolog: Backend Comparison . . . 45
5.8 The Price of Laziness: Per-Decision Cost . . . 49
5.9 An External Reference: Alpha . . . 53
5.10 The clingo Reading of Partial Information: `gc` and `lc` . . . 59
5.10.1 BSP: inertness . . . 59
5.10.2 PUP: the same inertness, with the same surcharge . . . 59
5.10.3 HRP: misguidance . . . 60
5.10.4 What the three cases share . . . 60
5.11 Two Ground Simulations of Alpha: `ga` versus `ga_weak` . . . 61
5.12 Evaluating Pre-Materialized Auxiliary Predicates . . . 63
5.13 Effect of the Constraint Formulation . . . 65
5.13.1 The full `_co` family: do the heuristics still matter? . . . 67

6 Discussion and Conclusions **70**
6.1 Discussion of the Results . . . 70
6.1.1 What lazy evaluation saves . . . 70
6.1.2 Why the semantics matters . . . 71
6.1.3 The runtime cost and the choice of backend . . . 71
6.1.4 What the comparison with Alpha establishes . . . 72
6.2 Limitations . . . 72
6.3 Implications . . . 73
6.4 Future Work . . . 74
6.5 Conclusions . . . 76

Bibliography **77**

Abstract

In the area of Answer Set Programming (ASP), ground-and-solve systems such as clingo require the instantiation of first-order rules before search begins. Although declarative domain-specific heuristics provide valuable search guidance, dynamic heuristics that depend on runtime search state (such as dynamic aggregates) suffer from combinatorial explosion during grounding. Conversely, lazy-grounding systems evaluate heuristics dynamically over partial assignments, but can incur additional search overhead from interleaving grounding and solving. This thesis introduces a lazy declarative heuristic framework for clingo using its propagator interface, combining dynamic heuristic evaluation with the efficiency of ground-and-solve systems.

Two execution backends are provided: a compiled C++ backend with incremental aggregate state maintenance, and an embedded SWI-Prolog engine with relational query evaluation over the solver trail, which it reads in place through foreign predicates. Our methodology decouples heuristic representation (ground vs. lazy) from interpretation semantics (clingo-like vs. Alpha-like) through order-preserving weight flattening and the appropriate evaluation of default negation on unassigned atoms. On the tested encodings, effective guidance depends on matching the intended reading of partial assignments. Benchmark results on three combinatorial problem families (the Balanced Sum Problem, the Partner Units Problem, and the House Reconfiguration Problem) show that lazy evaluation replaces aggregate-dependent ground heuristic directives with compact template facts and preserves effective search guidance. On the largest published PUP instance, lazy evaluation approximately halves grounding time and peak memory relative to the ground simulation. A compact reformulation of BSP makes the representation saving more pronounced. The comparison with the lazy-grounding solver Alpha shows that lazy clingo reaches similar peak process memory on the largest PUP instance, while remaining slower. On BSP, grounding the base program remains the dominant cost; on HRP, where that program is small, native lazy clingo is faster than the tested Alpha configurations.

Chapter 1

Introduction

Answer Set Programming (ASP) is a declarative approach to combinatorial search. Instead of describing an algorithm step by step, the modeller describes the properties that an admissible solution must satisfy, and a solver enumerates the assignments that satisfy them [1, 2, 3].

ASP solvers are essentially highly optimized backtracking, or more appropriately *backjumping*, engines. Modern ASP solving is dominated by two main paradigms: ground-and-solve and lazy grounding.

In ground-and-solve systems, such as clingo, first-order rules are instantiated over the problem domain before search starts, including the domain-specific heuristics that direct the solver's search. This approach can be severely memory-intensive, with grounding bottlenecks manifesting in two main scenarios: as problem instances scale, and whenever domain-specific heuristics depend on dynamic search state, for instance through an aggregate over the atoms that are true *at that point of the search*. Because grounding precedes search, the grounder must enumerate every possible value the aggregate could take, and a single heuristic ends up producing a number of ground objects that is combinatorial in the size of the instance. In contrast, lazy-grounding systems interleave grounding and solving. This can reduce memory use and extend reach on memory-intensive problems, but interleaving grounding and solving can also make search slower.

Depending on its underlying architecture, an ASP solver can be seen as an engine that compiles or interprets a very convenient and elegant declarative language [4] to search for a solution. The convenience of that language is paid for in control: the modeller states *what* an admissible solution is, but not *how* the engine should look for one. A pure backtracking approach behaves as an uninformed search, and on problems whose structure is known by construction, guiding the solver with that knowledge is extremely helpful.

Domain-specific heuristics are the means by which the modeller feeds that knowledge into the search. Syntactically they are declarative annotations that the grounder instantiates like ordinary rules, and their purpose is to tell the solver which atom to decide next, and with which polarity; their impact on solving is well documented [5, 6]. They are, however, part of the program, and clingo instantiates the program before search starts. A heuristic is therefore expanded, in full, before the first decision is taken. That expansion can be relatively harmless as long as the

heuristic depends only on the input. It stops being harmless as soon as the heuristic depends on *how the search is going*, which is precisely when a heuristic is most useful. Consider the directive used throughout this thesis to guide the Balanced Sum Problem (BSP), the problem of splitting $\{1, \ldots, n\}$ into two sets `b/1` and `c/1` of nearly equal sum:

```
#heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)}, W = ((n+1)*S)+X. [W, true]
```

The rule gives a higher weight to placing X in b as the current sum S in c increases. The aggregate value S appears *inside the weight*, and the atoms `c(Y)` it sums over are guessed during search rather than given as input facts. In general, an aggregate over guessed atoms may have to admit up to $2^{|Y|}$ potential values of the sum in the worst case (for instance, if the domain elements are powers of two, $Y \in \{1, 2, 4, 8, \ldots\}$). Since the grounder cannot know what S will be at decision time, it must enumerate every reachable sum. Even in this benchmark, where the domain $\{1, \ldots, n\}$ restricts the potential values to the dense interval $S \in \{0, \ldots, \frac{n(n+1)}{2}\}$, the grounder must still instantiate a separate ground directive for every pair (X, S). This yields $\Theta(n^3)$ ground objects (over one million, namely 1,010,200, directives at $n = 100$) for the symmetric rules of $b/1$ and $c/1$. All of them instantiate the same single rule the modeller wrote, and all but the one matching the current partial sum are inert at any given decision. A domain-specific heuristic introduced to help the engine solve a problem quicker ends up putting the answer out of reach.

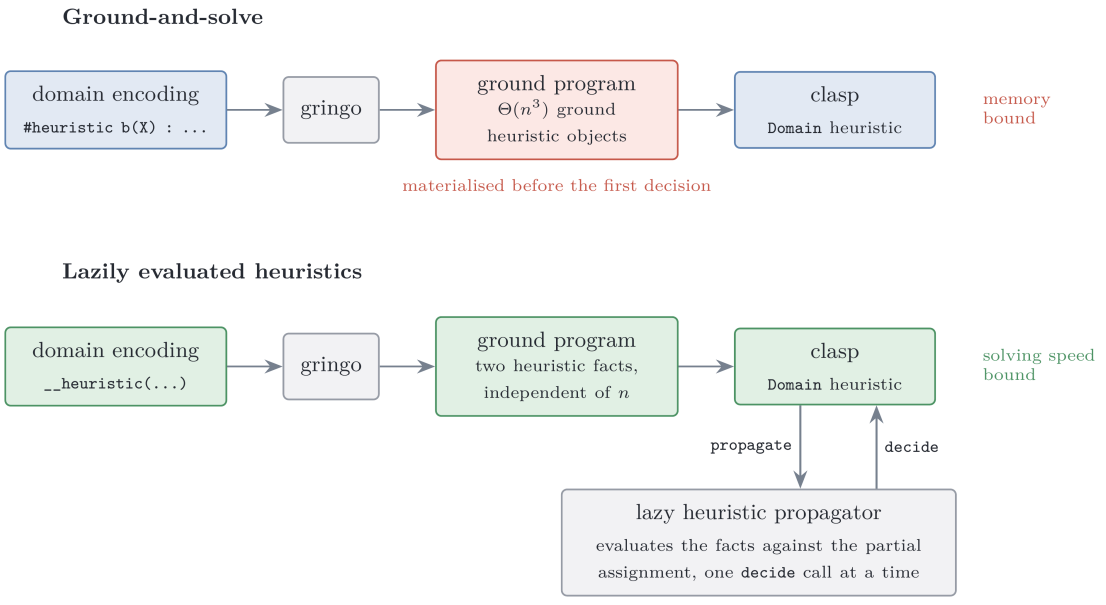


Figure 1.1: The same heuristic under the two approaches compared in this thesis.

The alternative is to leave the heuristic symbolic and to evaluate it while the search runs, when the partial assignment that S refers to actually exists. This is the natural approach in lazy-grounding solvers such as Alpha [7, 8, 9].

1.1 Objective and Contributions

This work asks whether lazily evaluated heuristics can be added to clingo without giving up its ground-and-solve pipeline (Figure 1.1), and whether the overhead of dynamic evaluation still allows the resulting system to outperform Alpha on the selected benchmarks.

The work contributes the following components and experimental analyses.

1. **A lazy heuristic evaluator for clingo:** A propagator, registered through the public `clingo::Heuristic` interface, that keeps a declarative heuristic in memory as a compact description and evaluates it at every decision against the current partial assignment.
2. **Two propagator backends:** A compiled C++ backend, which materializes target candidates and maintains incremental aggregate states, and an in-process SWI-Prolog backend, which evaluates the heuristic as an ordinary logic program against the solver trail, read directly through foreign predicates.
3. **Semantics and ablation study:** Ground-and-Solve, Lazy, clingo semantics and Alpha semantics combined give four variants of every encoding plus a heuristic-free baseline, to distinguish the effects of heuristic representation and interpretation.
4. **Weight translation from Alpha's syntax:** Alpha's priority levels are global, while clingo's are local to a single atom. Section 3.1.3 presents the transformation that folds a level into the weight and the conditions under which it preserves the intended order; Section 3.1.4 the weight field limit imposed by clingo.
5. **Trade-off analysis of the lazy approach:** By collecting per-decision metrics in both backends, we measure the runtime overhead of lazy heuristic evaluation against its savings in program size.

The evaluation covers three problem families, the Balanced Sum Problem (BSP), the Partner Units Problem (PUP), and the House Reconfiguration Problem (HRP), on both backends. The implementation is publicly available.[1]

1.2 Thesis Organisation

Chapter 2 introduces the foundations and related work. Chapter 3 describes the methodology and experimental design. Chapter 4 presents the implementation and benchmark setup. Chapter 5 reports the experimental results. Chapter 6 discusses the findings, limitations, and directions for future work.

[1] `https://github.com/pierspad/clingo-lazy-heuristic-evaluator`

Chapter 2

Background

2.1 Answer Set Programming

A *normal rule* in ASP has the form

$$a \;\leftarrow\; b_1, \ldots, b_m,\; not\; c_1, \ldots, not\; c_n,$$

where a, the b_i and the c_j are atoms: the head a is derived whenever every positive body atom b_i is derivable and no negative body atom c_j holds. The negation *not* is *default negation*: *not* c holds in the absence of a derivation for c. Because a rule read this way can justify itself, the meaning of a program is not given by the rules alone but through the notion of a stable model. Fix a set of atoms X, the candidate solution, and a ground program P. The *reduct* P^X is obtained by deleting every rule whose negative body intersects X, and by deleting the negative literals from the rules that survive. The reduct is a positive program, so it has a unique least model, and X is a *stable model* (or *answer set*) of P exactly when it coincides with that least model [1]. On the two-rule program

$$\begin{Bmatrix} a \leftarrow not\; b \\ b \leftarrow not\; a \end{Bmatrix}$$

the candidate $X = \{a\}$ gives the reduct $\{a \leftarrow\}$, whose least model is $\{a\}$: stable. The candidate $\{a, b\}$ deletes both rules, leaving the least model $\emptyset$: not stable. A stable model is then a set of atoms both closed under the rules and justified by them, in which every atom has a non-circular derivation that survives the model's own assumptions about what is false.

In the *guess-and-check* idiom a choice construct opens the solution space, integrity constraints prune the candidates that violate the specification, and the answer sets of the program correspond one-to-one to the solutions of the problem. ASP syntax extends normal rules with first-order variables, arithmetic, conditional literals, and three constructs this thesis uses throughout. A *choice rule* such as `{ b(X) } :- x(X).` permits any subset of its head atoms, introducing alternatives that the solver may resolve through propagation or decisions. An *integrity constraint* `:- Body.` is a rule with empty head and forbids every model that satisfies *Body*. An *aggregate atom* such as `#sum`, `#count`, `#min` or `#max` condenses a set of weighted

contributions into a single comparable value [3]. Aggregates are central to this work: the heuristics under study rank decisions by expressions such as "the current sum of the elements already placed in a set", and how and *when* such an aggregate is evaluated is exactly what differentiates the systems compared here.

2.2 Ground-and-Solve: gringo, clasp, and Conflict-Driven Search

clingo [3] couples the grounder gringo with the solver clasp into a single system. The grounder instantiates the first-order program over its Herbrand universe, producing a variable-free program. To mitigate combinatorial explosion, gringo restricts this expansion to rule instances whose positive body is potentially derivable; as a result, the ground program is generally much smaller than a full instantiation, though it must still be materialized in full before search starts. The solver clasp [10] then searches for the stable models of the ground program using *conflict-driven nogood learning* (CDNL), an algorithm that learns from conflict-inducing choices. The solver maintains a *partial assignment*, a consistent set of literals over the program atoms; it alternates *unit propagation* steps with *decision* steps, and when propagation derives a contradiction, it analyzes the conflict, learns a nogood built around the *First Unique Implication Point* (1UIP), and *backjumps* to the decision level at which that nogood becomes unit. Which atom to decide on, and with which polarity, is delegated to a *decision heuristic*; clasp's default is activity-based, in the family introduced by Moskewicz et al. [11], and favors atoms that appear in recent conflicts.

In the single-shot setting considered here, the ground input program is materialized before search begins; additional nogoods are learned during solving. The *grounding bottleneck* is a well-known limitation of ground-and-solve systems [8]; Chapter 5 measures one concrete manifestation of it, namely heuristic directives whose weight depends on an aggregate value.

2.3 Domain-Specific Heuristics in clingo

clasp allows the modeller to steer its decision heuristic using domain-specific knowledge through the `#heuristic` directive [5]:

```
1 #heuristic A : Body. [W@P, M]
```

Here, A is the target domain atom whose decision choice is steered according to the modifier M (e.g., setting its sign or priority level). The heuristic applies only if the rule *Body* is satisfied. If multiple heuristic directives target the same atom A, clingo selects the one with the highest priority P to determine the atom's modifier and weight. Across distinct target atoms, clasp then prioritizes candidates with higher weight W. Alpha reads the same two fields the other way round, as the next section describes.

2.4 Lazy Grounding, Alpha, and Heuristics over Partial Assignments

Lazy-grounding solvers invert the order of the two phases. Rather than instantiating the program and then searching it, they instantiate a rule only when the partial assignment reached so far makes it relevant, avoiding the need to materialize the full ground program before search. Exploring unsuccessful branches can, however, require additional rule instantiation and nogood generation. Alpha [7] is the external lazy-grounding reference used in this work.

Comploi-Taupe et al. extended Alpha with declaratively specified domain-specific heuristics evaluated against the *partial* assignment maintained during solving [8], and subsequently with aggregates evaluated over that same partial assignment [9]. Because a heuristic is read while the assignment is still incomplete, its semantics differs from clingo's on the following points:

- **Default negation.** `not a` holds as soon as a is not assigned true, rather than only once a has been assigned false. A guard over an atom the search has not yet reached is therefore open in Alpha and closed in clingo.
- **Dynamic aggregates.** An aggregate is evaluated over its currently true source atoms. For example, if only `c(2)` and `c(5)` are true, `S = #sum{Y : c(Y)}` yields $S = 7$, even while other `c/1` atoms remain unassigned. The heuristic can use this partial sum immediately and update its weight as the assignment changes. In clingo, the same equality binding is grounded over candidate values, and its satisfaction during search follows the ordinary aggregate semantics; it does not simply bind S to the sum of the currently true atoms.
- **Rank.** The annotation $[W@P]$ is an absolute rank over all heuristic instances: the level P decides first, always preferring higher values, and the weight W orders only the instances that share a level. In clingo, as Section 2.3 recorded, P arbitrates between directives on the *same* target atom and W alone orders distinct atoms.

An encoding written for one system therefore cannot be handed to the other verbatim. Chapter 3 describes how these differences are handled within the two axes of the experimental design: heuristic representation and semantics.

2.5 Extending clingo: Propagators and the Heuristic Interface

Standard clingo evaluates grounded heuristic conditions during search, but does not evaluate the symbolic aggregate templates over the currently true atoms in the way required here. It also provides an extension mechanism: *theory propagators*, user-defined components registered with the solver that participate directly in the search loop [12]. In this work, that interface computes heuristic guidance from the current solver trail.

A propagator has four callbacks. During `init`, it inspects the symbolic atoms of the ground program, maps the atoms it cares about to solver literals, and regis-

ters *watches* on them. During search, `propagate` is invoked when a watched literal becomes true, `undo` when backtracking retracts it, and `check` according to the configured check mode, by default on total assignments. Watches trigger propagation callbacks; checks and heuristic decisions have their own invocation points.

The `clingo::Heuristic` interface inherits from Propagator and extends it with a `decide` callback, invoked at each decision step *before* the solver's built-in heuristic. The Heuristic object receives the current assignment of the solver and either returns a signed literal, which becomes the next decision, or returns 0, in which case clasp falls back to its default activity-based heuristic (VSIDS [11]). This is the mechanism the rest of the thesis builds on. It keeps the declarative heuristic descriptions in memory, evaluates them incrementally against the trail, and answers each `decide` call with the best applicable target.

Chapter 3

Methodology

3.1 The Experimental Design: Two Axes, Four Variants

The main experiment compares encodings of the same problem that differ in how the domain-specific heuristic is represented and interpreted. Within each family, the problem constraints are held fixed while these two properties vary. The exploratory variants separately examine auxiliary predicates and, on BSP, a compact reformulation of the balance constraint.

The *approach* axis fixes where the heuristic is evaluated. In the ground-and-solve approach the heuristic is written with native `#heuristic` directives: gringo expands them before search and clasp's Domain heuristic consumes the resulting ground objects. In the lazy approach the heuristic remains a handful of ground facts, `__heuristic(...)` for the native backend and `heuristic("...")` for the Prolog one, which the propagator reads at its initialization and evaluates at every decision step against the partial assignment.

The *semantics* axis fixes how a heuristic condition reads partial information. Under the clingo-like reading, `not a` requires a to be assigned false; under the Alpha-like reading, it holds whenever a is not assigned true. Alpha-like aggregates are computed over the atoms currently true, allowing the BSP heuristic to compare the two partial sums during search. The lazy clingo-like variant instead waits until all aggregate sources are assigned. This is the policy implemented for aggregate bindings in our evaluator; clingo itself can determine some aggregate conditions, such as monotone bounds, earlier.

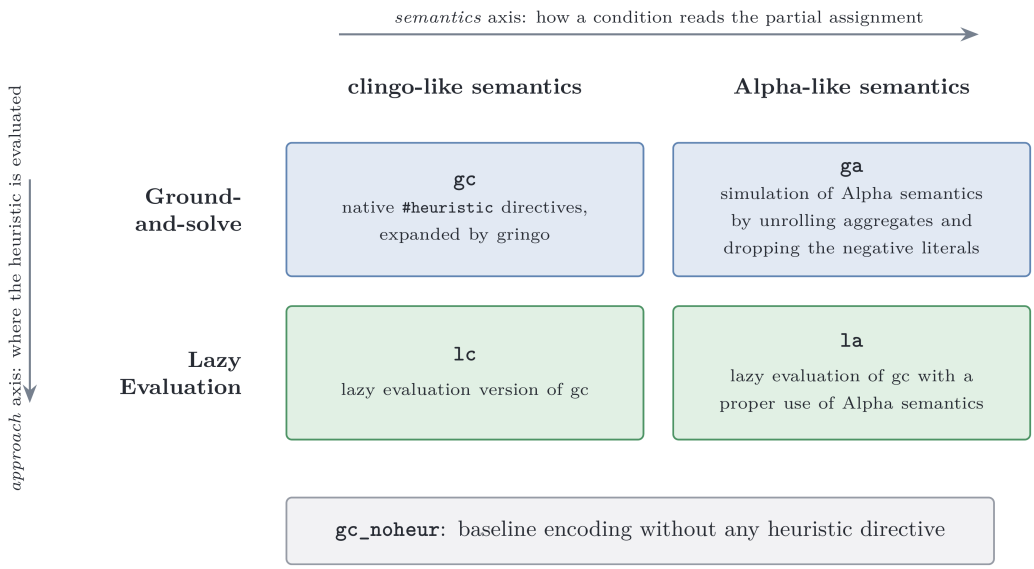


Figure 3.1: The experimental matrix. Prefixes name the approach, suffixes the semantics.

Crossing the two axes yields the four variants that the rest of the thesis refers to as: `gc` and `ga` for the ground approach under the clingo and the Alpha semantics, `lc` and `la` for the lazily evaluated one, the prefix naming the approach and the suffix the semantics. To them is added the heuristic-free baseline `gc_noheur`, which carries the same domain logic with no heuristic at all and measures what the solver does when it is left alone (Figure 3.1).

To isolate specific architectural and modeling mechanisms beyond this primary matrix, the experimental design also incorporates two exploratory variants, `ga_weak` and `la_aux`, and a second set of BSP encodings, the `_co` family, covering the four main variants, the heuristic-free baseline, and `ga_weak`. Here, the *BSP matrix* denotes this collection of experimental configurations for the Balanced Sum Problem. The `_co` family reformulates the problem's balance constraint, which requires the sums of the two partitions to differ by at most one. This is a constraint defining admissible solutions, rather than a condition controlling heuristic applicability. The empirical motivation and methodological role of these exploratory variants in disentangling the contributions of default negation, body materialization, and constraint formulation are discussed in Section 3.1.2.

3.1.1 Simulating Alpha semantics

Reproducing an Alpha-like behaviour within clingo requires addressing three distinct aspects: default negation over unassigned literals, dynamic aggregate evaluation, and the value an aggregate takes before its sources exist.

Default negation over unassigned literals

The first transformation concerns negative literals in heuristic bodies: `not c(X)` in BSP, `not not_assigned_sensor_unit(S,U)` in PUP, and the guards `not`

`fullCabinet(C)` and `not assignedThing(T)` in HRP.

clasp applies heuristic modifications only to variables that are currently unassigned, as enforced in `DomainHeuristic::propagate`:

```
if (s.value(a.var) == value_free && a.prio >= prio) { applyAction(s, a, prio);
↪ }
```

Here, `s` is the solver and `a` is the heuristic action associated with a satisfied condition. The field `a.var` identifies its target variable; `s.value(a.var) == value_free` checks that this variable is still unassigned. The comparison `a.prio >= prio` checks the action's priority against the priority currently stored for the same target variable and modifier. Only when both checks succeed does `applyAction` apply the heuristic modification. It follows that *a directive whose body entails the truth value of its own target atom can never fire*: once the body conditions hold, the target is already decided, and clasp skips it.

All heuristic directives in the suite that test the search state through negative literals show the same problem:

- In **BSP**, `b(X)` and `c(X)` are complementary choices, so the condition `not b(X)` is satisfied only once the target `c(X)` is already derived.
- In **PUP**, the heuristic body repeats the negative literal of the rule that derives the target atom, namely `not not_assigned_sensor_unit(S,U)`.
- In **HRP**, the placement directives for `cabinetTOthing(C,T)` require the guard `not assignedThing(T)`; in ground clingo this holds only when the candidate assignments for T are already proven false, meaning the target is never unassigned (`value_free`) when the directive evaluates.

Consequently, none of these directives ever fires in the `gc` variant. In HRP the same encoding also carries directives with no condition at all, which are applied at initialization and do steer the search; the conditional ones listed above stay inert.

The ground variant `ga` therefore drops these negative literals from the heuristic bodies. For directives where the negated literal simply tests the target atom or its complement (as in BSP and PUP), this preserves the intended applicability on unassigned targets, and clasp's `value_free` check still prevents modifying an already assigned atom. In HRP, however, the dropped literals also include capacity guards such as `not fullCabinet(C)`, which describe a shared resource rather than the target atom itself. There, dropping the guard is a static approximation used by this translation: it enables the directive to fire on unassigned atoms from the start of search, at the price of never deactivating once a cabinet becomes full.

In short, deleting `not a` from a heuristic body preserves the Alpha reading under the sufficient condition that a is the target atom or its complement. For other atoms, such as the HRP capacity guards, this argument does not establish equivalence; removing the guard can change which decisions the heuristic proposes. The HRP translation is therefore not established as equivalent to the Alpha-like heuristic. A treatment that preserves the guards' intended effect using clingo priorities remains future work.

Dynamic aggregates by bound-guarded unrolling

The second transformation replaces dynamic aggregate equality with bound-guarded unrolling over all candidate values. For BSP:

```
sum_value(0..tot).
#heuristic b(X) : x(X), sum_value(S), S <= #sum{Y : c(Y)},
    W = ((n+1)*S)+X. [W, true]
```

Here the directive is unrolled over all `sum_value(S)` and guarded by a bound condition, so that during search the currently satisfied bounds encode the current aggregate value, provided the guard is monotone, i.e. once satisfied it stays satisfied as further atoms become true.

Without explicit priorities all unrolled directives share the default priority 0. clasp applies newly satisfied directives by overwriting previous ones; it therefore retains the *most recently activated* directive, not the maximum-weight one. To match the reference heuristic, the unrolling must align the activation order with the weight order:

- **Lower-bound guards ($V \leq f$):** As f increases, the active set $\{V : V \leq f_{curr}\}$ expands upwards. The last activated value is $V = f_{curr}$, which yields the maximum weight if $W(V)$ is strictly *increasing* (e.g. in BSP, where $W(S) = (n{+}1)S + X$).
- **Upper-bound guards ($V \geq f$):** The active set $\{V : V \geq f_{curr}\}$ instead expands downwards, as f decreases, and the alignment requires $W(V)$ to be strictly *decreasing*.

Empty aggregates

`#max` over an empty set is 0 in Alpha but $-\infty$ (`#inf`) in clingo, and an aggregate read on a partial assignment is empty exactly at the start of search, when the heuristic is most needed. All encodings of the suite therefore write `#max{0; ...}` with an explicit neutral element, and both lazy backends implement the same rule: the native `MaxState` returns 0 when empty, and the Prolog runtime defines `dyn_max` with a 0 default.

3.1.2 Methodological Rationale for the Exploratory Variants

While the four main variants evaluate the core matrix of approach versus semantics, the following are introduced as experimental controls to isolate specific factors:

Isolating Alpha-Negation from Dynamic Aggregates (`ga_weak`). Alpha-like semantics treat default negation as satisfied over unassigned literals, and evaluate aggregates dynamically over currently true atoms. While `ga` simulates both via extensive ground unrolling, `ga_weak` acts as an ablation study that adapts the negative guards while retaining aggregate equality bindings instead of the monotone bounds used by `ga`. This tests whether adapting default negation alone improves search on these encodings, and what further benefit comes from dynamic aggregate evaluation.

Fewer directives, or less grounding? (`la_aux`). The `la_aux` variant retains the lazy propagator but expresses the BSP heuristic conditions as ordinary ASP rules. For the target `b(X)`, the condition is rewritten as:

```
h_b(X,S) :- x(X), not c(X), S = #sum{Y : c(Y)}.
```

A rule defines `h_c(X,S)`. At decision time, the lazy evaluator queries the auxiliary atoms that hold, matches their first argument to the target element X, and uses their second argument S to compute the weight $(n+1)S + X$. Thus the conditions are instantiated during grounding and evaluated by the solver during search; grounding does not precompute their truth values for future assignments.

The encoding still contains two lazy template facts, one for each target predicate, but the auxiliary rules are grounded over candidate pairs (X, S). The small template count therefore does not imply a small ground representation of the heuristic conditions. These templates must be distinguished from native `#heuristic` directives and their ground instances. The variant explores the effect of materializing conditions while retaining lazy selection; it does not by itself establish an internal representation different from clasp's treatment of native heuristic bodies, or isolate representation cost from changes in condition applicability.

Disentangling Heuristic and Problem Constraint Bottlenecks (the `_co` family). In the baseline BSP encoding, the balance requirement is enforced by computing the sums of both partitions independently and verifying that their difference does not exceed one via an integrity constraint (`:- sumB(SB), sumC(SC), (SB - SC)**2 > 1`). Because the possible partition sums scale quadratically with n, joining all pairs of candidate sums during grounding triggers a severe combinatorial explosion (materializing over 52.8 million propositional rules at $n = 120$). Grounding the base constraints then dominates total runtime across the variants, making the cost of the heuristic layer harder to distinguish.

The compact (`_co`) transformation resolves this bottleneck by exploiting the invariant that the total sum of all elements is known a priori ($tot = \sum_{i=1}^{n} i$). Instead of generating auxiliary predicates for both sums and checking them quadratically, the balance condition is restated as a single aggregate constraint with constant interval bounds, requiring the sum of partition b to fall within $[\lfloor tot/2 \rfloor, \lceil tot/2 \rceil]$ (Section 4.3.1). This removes the cross-product join over candidate sums entirely, collapsing the base encoding by five orders of magnitude (from millions of rules to just a few hundred). As an experimental control, the `_co` family eliminates the base grounding overhead, cleanly isolating the representational cost of the heuristics and clarifying whether dynamic guidance remains beneficial when domain propagation is already efficient.

3.1.3 Weights, Priorities, and Flattening

Section 2.4 recorded that the two systems read the annotation $[W@P]$ of a heuristic differently. In Alpha, the pair is an *absolute* rank over all heuristic instances, read lexicographically, with P compared first: the priority P is a global level and decides

first, so no instance of level P is considered while an instance of a level $P' > P$ is still applicable, and the weight W only orders the instances that share the same level. In clingo, the evaluation is different: P is local, arbitrating only between heuristics relative to the *same* target atom, while the order in which distinct atoms are decided comes from the weight alone. An encoding written for Alpha therefore cannot be handed to clingo verbatim.

The intended order can nevertheless be recovered, by *flattening* the level into the weight:

$$W_{flat} \;=\; W + P \cdot M, \qquad M > \max W - \min W,$$

where the intra-level weights W range over $[\min W, \max W]$ and the level multiplier M is larger than that range. Under this condition $W + P \cdot M > W' + P' \cdot M$ holds whenever $P > P'$, regardless of the two individual weights, and reduces to $W > W'$ when $P = P'$: the total order induced on the atoms is then the one Alpha's absolute levels induce. When all weights are non-negative, which is the case in every encoding of this suite, $M > \max W$ suffices.

Each benchmark fixes its own M. In the PUP encodings M is the constant offset 100,000 added to the zone weights, against a maximum sensor weight of 26,220 on the benchmark instances; in HRP M is derived from the instance as `levelMul(LM) :- LM = MC + MR + 10` over the maximum cabinet and room identifiers, which gives 16 on the smallest instance and 54 on the largest. BSP does not need a multiplier, as discussed in Section 4.3.1.

3.1.4 The Bounded Weight Field

clasp stores the weight of a `#heuristic` modification in a signed 16-bit integer field and clamps any value exceeding 32,767. Consequently, two ground directives whose weights both exceed 32,767 become indistinguishable to clasp's Domain heuristic. A flattened weight must therefore satisfy $W + P \cdot M \leq 32{,}767$ for the ground variants to rank as intended. HRP stays far below this bound; PUP exceeds it on the zone directives, with the consequences discussed in Section 5.5; and BSP, whose maximum weight grows as $\Theta(n^3)$, crosses it within the benchmarked range. What matters there is not the largest weight the grounder emits, but the largest one the search actually reaches: directives whose aggregate value is never attained may clamp without any observable effect. On the tested BSP sizes, the ground simulation matches the lazy search up to $n = 50$, deteriorates at $n = 60$, and times out from $n = 70$, as reported in Section 5.11.

In the compact `_co` encoding (Section 5.13.1), weights saturate in exactly the same way, yet runtime is unaffected because propagation resolves choices directly without needing heuristic guidance. The lazy backends avoid this limitation altogether. Rather than assigning static weights to clasp, the propagator ranks candidate targets dynamically at runtime and directly returns the chosen decision literal.

The suite folds each benchmark's global levels into the weights in all four variants and both backends, so cross-atom ordering does not depend on backend-specific priority semantics. Accordingly, the native lazy language (Section 3.3) omits a separate priority field. The Prolog representation retains one for interface compatibility:

BSP and PUP use `P=0`, whereas HRP assigns `P=1` to its nine ranking rules and `P=0` to the final false layer, as detailed in Section 3.3.3.

3.2 System Architecture

The project maintains two modified copies of clingo. They share the same upstream sources and differ only in their extensions: the build in `clingo-native` carries the C++ backend, while the build in `clingo-prolog` carries the Prolog backend, configured with `clingo_USE_SWIPL=ON` so that the SWI-Prolog engine is linked in-process, making its memory impact easier to account for. In both builds, the extension lives entirely under `libclingo`, exposes the same class name, and is registered at the same point of `clingo_app.cc`:

```
1 HeuristicPropagator my_propagator;
2 ctl.register_propagator(my_propagator, true);
```

The propagator is registered on *every* run of the modified binary. On a program that contains no lazy heuristic facts, the propagator itself is inactive: `init` finds nothing to watch and `decide` always returns 0, so every decision is left to clasp's own branching heuristic. The same binary therefore also runs the ground variants (with clingo's standard `#heuristic`s) and the heuristic-free baseline without requiring a separate build.

In the registration call `ctl.register_propagator(my_propagator, true)` shown above, the second argument, `true`, instructs clasp to call the propagator sequentially, avoiding race conditions on the incremental state described in Section 4.1.1. No other parts of clingo are modified. Figure 3.2 shows the architecture of the two binary build variants.

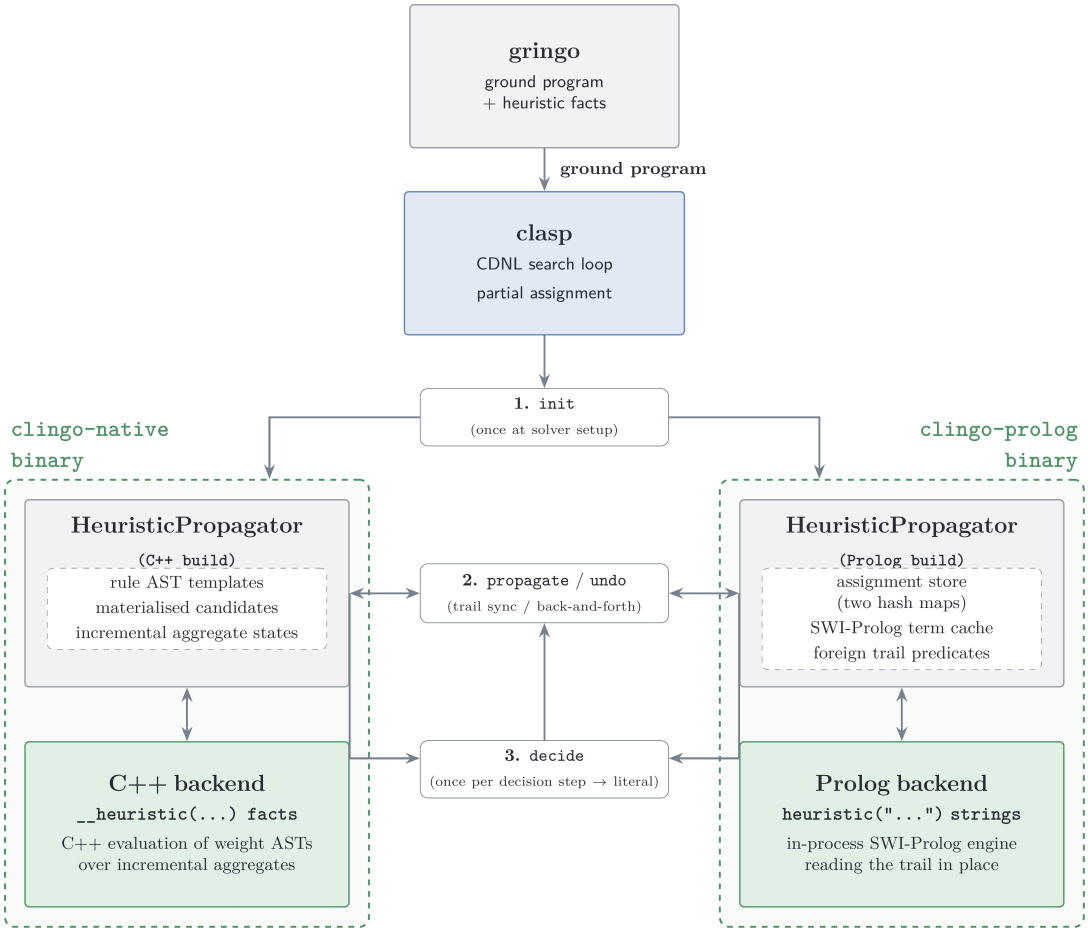


Figure 3.2: System architecture of the two binary variants, showing their respective heuristic propagators and evaluation backends attached to clasp.

In both builds, the class implements `clingo::Heuristic`, the propagator interface extended with the `decide` callback (Section 2.5). Both backends follow the same lifecycle, built on `init`, `propagate`, `undo` and `decide`:

- During `init`, they scan symbolic atoms for trigger facts (`__heuristic(...)` in the C++ build and `heuristic("...")` in the Prolog build), construct their runtime representation, and register watches on solver literals.
- During forward search, `propagate` updates the internal state of the aggregates appearing in the heuristic bodies, together with the candidate states, as watched literals become true.
- During backtracking, `undo` restores those aggregate and candidate states as literals are retracted from the trail.
- At each decision point, `decide` either returns a signed literal for the highest-ranked target or returns 0, delegating the decision to clasp's fallback heuristic.

The two backends support different heuristic languages. The native backend maintains an incremental state for `#sum`, `#count`, `#min` and `#max` over one predicate and

one argument position, and can only filter the aggregated set by matching its arguments with those of the target atom; the Prolog backend evaluates an arbitrary `aggregate_all/3` goal and is unrestricted in this respect. Section 3.3 states the language subset each backend accepts, and Section 3.3.3 what falls outside it.

To activate this mechanism, runs are launched with the flag `-heuristic=Domain`. A detailed discussion of the architectural differences between the two backends is provided in Chapter 4.

The propagator never adds a constraint, never derives a literal, and never refuses one. It changes only the order and preferred polarity of decision literals, so the answer sets are untouched; a heuristic can make a run slower or faster but cannot change the meaning of the encoding (detailed in Section 4.1.3).

3.3 Translation of the Heuristic Directives

Neither backend reads native `#heuristic` directives directly. Instead, a custom declarative language replaces each directive with a single ground fact that *describes* the heuristic rule.

This substitution is driven by two main reasons:

- gringo treats a `#heuristic` directive as an ordinary ASP rule and immediately grounds it. By the time the propagator initializes in `init`, the combinatorial explosion has already occurred. What the propagator needs is the high-level *template*, which gringo never grounds and the clingo C++ API does not expose; writing it as an ordinary ground fact is what makes it reachable from `init`.
- Aggregates must survive grounding unevaluated so they can be monitored dynamically during search. Standard ASP syntax provides no such mechanism: an aggregate inside a rule body is either evaluated statically by the grounder or unrolled over its value domain.

The lazy fact is therefore the original directive rewritten in a form that passes through the grounder untouched. Its structure mirrors the original construct-by-construct. The remainder of this section details this representation. We introduce each construct by rewriting the BSP heuristic (Chapter 1) step by step. Finally, Section 3.3.3 outlines the expressive limitations of both backends.

3.3.1 Lazy heuristic syntax for the C++ backend

```
1 #heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)}, W = ((n+1)*S)+X. [W,
  ↪  true]
```

Target. The target atom is `b(X)`, where X ranges over the ground instances. The lazy fact specifies only the predicate name, `__target(b)`. During `init`, the propagator queries clingo's atom table for all ground instances of `b(...)` already materialized by gringo.

Positive body. The condition `x(X)` shares its argument with the target: the candidate for `b(7)` is alive only while `x(7)` is true. A condition of this shape is written as the bare predicate name, `x`, and the propagator matches its arguments against the target tuple position by position.

If a body predicate has a different arity or argument order, or supplies a value used in the weight, the correspondence must be stated explicitly:

```
__body(pred, __match(src, tgt), __bind_arg(var, idx))
```

`__match(src, tgt)` requires the argument at the `src` position of the body atom to equal the argument at the `tgt` position of the target atom, where both `src` and `tgt` are integers. All indices are 0-based, and at least one `__match` is mandatory. `__bind_arg(var, idx)` extracts argument `idx` of the body atom into a variable.

Given the ground facts:

```
score(1,10). score(2,5). score(3,7).
```

and the target atoms:

```
{ choose(1..3) }.
```

the directive:

```
__heuristic(__target(choose), __body(score, __match(0,0), __bind_arg(w,1)),
↪  __weight(w), __modifier(true)).
```

pairs each `choose(X)` with the `score(X,W)` that agrees with it on the first argument, and binds `w` to the second argument of that atom. The weights are then 10, 5 and 7, so the propagator decides `choose(1)` first.

Negative body. The condition `not c(X)` likewise matches its arguments against the target tuple. Negative body literals are specified by prefixing the predicate name with `__n_`.

Semantics. The reading of that negation is fixed by the `__semantics(S)` selector, which decides whether `__n_c` means "`c(X)` is assigned false" or "`c(X)` is not assigned true". The same selector fixes the reading of aggregates, described below. It takes `alpha` or `clingo`, and defaults to `clingo`, selecting the clingo-like reading of body conditions described above.

Aggregates. Aggregates are evaluated lazily during search. For instance, `__bind(s, __sum(c))` binds variable `s` to the sum over the predicate `c` (by default aggregating its first argument). The operators `__count`, `__min`, and `__max` follow the same structure.

An aggregate can be restricted using `__filter(src_idx, tgt_idx, offset)`, which narrows it *per target*, keeping only source atoms whose argument at position

`src_idx` equals argument `tgt_idx` of the target plus an optional `offset`. For instance, in PUP, retrieving the maximum number of sensors on the *previous* unit $(U-1)$ for a target `assigned_zone_unit(Z,U)` is expressed as:

```
__bind(m1, __max(num_sensors_on_unit, 0, __filter(1, 1, -1)))
```

where `0` selects the aggregated argument N in `num_sensors_on_unit(N, U')`, and the filter matches $U' = U - 1$.

Weight. Weight is expressed as a *term tree* such as:

```
__weight(__add(__mul(s, B), self))
```

which represents $(s \cdot B + self)$, where `self` is a shorthand for the target atom's argument at index 0 without requiring a variable binding.

The binary constructors `__add`, `__sub`, and `__mul` form expression trees whose leaves belong to three categories: integer constants, variables of the heuristic language, and target arguments (accessed through `self` or `__bind_target_arg(var, idx)` for any other).

At `init`, the propagator compiles all weight term trees into ASTs. During search, each active heuristic object evaluates its weight AST dynamically against the current trail state, avoiding the memory overhead of materializing them in the ground program. Only candidates with a strictly positive resolved weight are taken into account; the others are ignored, leaving their decision to the solver's default heuristic.

Modifier. The `__modifier(M)` selector specifies which of clingo's heuristic modifiers to use: `level` sets the ranking value alone, `sign` sets the preferred polarity alone, while `true` (the default) and `false` set both.

The solver's `decide` interface expects a *signed* literal, so a ranking value alone does not determine a decision. Each target keeps one slot per channel, ranking value and preferred polarity, and when several directives write to the same slot the one with the highest weight wins. This is what replaces clingo's local priority. Across the benchmarks all directives use `true`, except for the final layer of HRP, which uses `false` to prune the remaining choice atoms.

Assembling the components gives the fact in full:

```
__heuristic(__target(b),
            x,
            __n_c,
            __bind(s, __sum(c)),
            __weight(__add(__mul(s, B), self)),
            __semantics(alpha),
            __modifier(true))
                :- B = n + 1.
```

3.3.2 Lazy heuristic syntax for the Prolog backend

Unlike the native backend, which supports a fixed heuristic DSL, the Prolog backend accepts standard goals available to the embedded engine. A heuristic is declared as a string inside a `heuristic("...")` fact and parsed directly as a Prolog clause.

```
heuristic(b(X), W, 0, true) :-
    holds(x(X)),
    target_available(b(X)),
    alpha_not(c(X)),
    aggregate_all(sum(Y), holds(c(Y)), S),
    holds(heuristic_base(Base)),
    W is Base * S + X.
```

The **head** specifies the target atom (A), weight (W), priority (P) and modifier (M). The **body** gives the conditions and computations used to derive them. Priorities follow the convention described in Section 3.1.4.

To interact with the solver's search state, the backend provides a few dedicated interface predicates: `holds/1` succeeds on atoms currently true on the trail and on static ground facts; `alpha_not/1` and `clingo_not/1` implement the two readings of default negation described in Section 2.4; and `target_available/1` discards candidates that are already assigned.

Everything else uses standard Prolog: arithmetic relies on `is/2`, and set aggregations use `aggregate_all/3`. The backend also provides the wrappers `dyn_sum/3`, `dyn_count/2`, `dyn_min/3`, and `dyn_max/3`. When no matching atoms are assigned on the trail, `dyn_min/3` and `dyn_max/3` return 0 instead of failing. This lets the heuristic compute a weight even at the start of search.

Table 3.1 summarizes the correspondence between the three notations.

construct	clingo directive	native lazy fact	Prolog rule string
structure			
target	`b(X)`	`__target(b)`	`heuristic(b(X), W, P, M)`
positive body, target tuple	`x(X)`	`x`	`holds(x(X))`
positive body, join	`p(X,V)`	`__body(p, __match(0,0), __bind_arg(v,1))`	`holds(p(X,V))`
negation	`not c(X)`	`__n_c`	`alpha_not(c(X))` `clingo_not(c(X))`
aggregate	`S = #sum{Y : c(Y)}`	`__bind(s, __sum(c))` also `__count`, `__min`, `__max`	`aggregate_all( sum(Y), holds(c(Y)), S)`
filtered aggregate	`M = #max{N : q(N,U-1)}`	`__bind(m, __max(q, 0, __filter(1,1,-1)))`	`U1 is U-1, dyn_max( holds(q(N,U1)), N, M)`
target arguments	`X`, `Y` in `b(X,Y)`	`self`, `__bind_target_arg( y, 1)`	ordinary head variables
annotations			
weight	`W = ((n+1)*S)+X`	`__weight(__add( __mul(s,B), self))`	`W is B*S+X`
modifier	`[W, true]`	`__modifier(true)`, default `true`	fourth head argument
semantics	(fixed by the system)	`__semantics(alpha)` `__semantics(clingo)`	choice of `alpha_not` / `clingo_not`
target still free	(implicit in clasp)	implicit: an assigned target is skipped	`target_available( b(X))`

Table 3.1: The same heuristic in the three notations.

3.3.3 What the Two Languages Cannot Express

While Table 3.1 summarizes the syntax supported by each backend, the two engines do not cover the same subset of clingo's heuristic language, nor does either support standard ASP heuristics in full. Instead, they reflect a clear design trade-off between evaluation speed and relational expressiveness.

The C++ language subset. The C++ backend uses a restricted template representation to study incremental heuristic evaluation in a preliminary prototype. It provides a native implementation against which the cost of the more expressive Prolog evaluator can be measured; the comparison concerns the complete implementations, not an isolated cost of the Prolog language. The template is not intended as

a convenient end-user syntax. A future translator could map a supported fragment of clingo heuristic directives into this representation, while preserving the explicitly selected evaluation policy (Section 6.4). Directives must target entire predicates rather than individual ground atoms, and arguments are strictly limited to numeric tuples.

In ordinary rule notation, supported body conditions link directly to the target tuple: for a target `choose(X)`, a condition `score(X,W)` can match on X and bind W for the weight. A body-only variable cannot implement an arbitrary join between conditions, as in `p(X,V), q(V,W)`. Such joins and relational comparisons (`N < M`) require auxiliary ASP rules outside the template. Aggregates range over one source predicate and aggregate argument, with the target-argument matching filters described above. For example, `#sum{Y : c(Y)}` is supported; an aggregate condition joining several predicates is not directly supported.

Finally, weight arithmetic is restricted to basic operations ($+$, $-$, $\cdot$) yielding strictly positive values, and modifiers altering internal VSIDS scores (`init`, `factor`) are rejected. Because the backend also omits an explicit priority field, multi-level priority hierarchies must be flattened into weights (Section 3.1.3).

The Prolog language subset. In contrast, the Prolog backend prioritizes expressive flexibility. Because rule bodies are standard Prolog goals, arbitrary joins, comparisons, nested aggregations (via `aggregate_all/3`), and non-numeric terms are supported natively. Its head also carries an explicit priority argument, so it *can* express Alpha's two-dimensional rank without weight flattening; the suite uses flattened weights throughout, and the priority field only in HRP, where it keeps the nine ranking rules one level above the final `false` layer. A separate integration requirement is that a generated target must correspond to an atom with a solver literal. Even when all variables are bound, a Prolog goal can construct a ground target term absent from that table. The benchmark rules use `target_available/1` to restrict targets to known, unassigned atoms; the backend also checks proposed targets before returning a decision (Section 4.2.3). Unrestricted goals are re-evaluated from scratch at every decision point, introducing runtime overhead that is evaluated in Section 5.8.

Chapter 4

Implementation and Benchmark Setup

4.1 Inside the Native Backend

4.1.1 Parser and Runtime Representation

The parser in `libclingo/src/heuristic_parser.cc` turns each `__heuristic` symbol into a `HeuristicRuleTemplate`: target atom, positive and negative body literals, variable bindings, modifier, semantics, and an arithmetic AST for the weight.

During `init`, the propagator inspects the ground symbolic atoms and builds an index mapping each predicate and its argument tuple to the corresponding solver literal. It then materializes candidates: for each ground target atom and matching combination of positive body atoms, it creates a `RuntimeHeuristicCandidate` storing the target literal, target tuple, body literals, bound variable values, and instantiated aggregate keys.

In BSP with $n = 100$, this yields only 100 candidates for `b(X)` (and 200 across both partitions), whereas the standard ground encoding generates 1,010,200 directives ($2 \times 100 \times 5051$) to cover all possible discrete sums $S \in \{0, \ldots, 5050\}$. The target atoms themselves are still grounded by gringo as usual: what the propagator avoids is not the grounding of heuristics per se, but the enumeration of the heuristic directive over the domain of the aggregate. For this direct BSP formulation, candidate materialization therefore scales linearly with the number of ground targets rather than with the domain size of dynamic aggregates, and the aggregate value is computed at search time instead of being fixed by a ground directive. This is a narrower form of laziness than Alpha, which instantiates the whole program on demand.

Finally, the propagator registers watches exclusively for literals that affect heuristic state: both polarities of target and negative body literals, positive body literals, and aggregate source literals.

4.1.2 Incremental Aggregates

Aggregate states implement three methods: `add`, `remove`, and `result`.

Sums and counts (`SumState`, `CountState`) store a single `int`, because adding and removing values can be done with simple addition and subtraction. In contrast, min and max (`MinState`, `MaxState`) cannot be undone with arithmetic; when backtracking removes the current minimum or maximum, the state must recover the next active value. They therefore use an `std::map<int, int>` that maps each active value to how many times it occurs.

Each source atom updates the `AggregateState` matched by its `Runtime AggregateKey`: values are added in `propagate` and removed in `undo`.

In the native clingo-like mode, an aggregate binding is evaluated only after all its source literals are assigned, gating dependent candidates until then. Under Alpha semantics, candidates directly read whatever partial aggregate value is currently available on the trail.

4.1.3 Evaluation and Decision

A candidate is *active* when its target literal remains unassigned, all its positive body literals evaluate to true, and every negative body literal is satisfied under the template's semantics. For negative literals, the two supported semantics use the following test:

```
static bool negative_literal_is_satisfied(HeuristicSemantics s,
↪  clingo::TruthValue v) {
    return s == HeuristicSemantics::clingo ? v == clingo::TruthValue::False
                                           : v != clingo::TruthValue::True;
}
```

Once a candidate satisfies its body conditions, the propagator computes its numerical weight by evaluating its weight AST over the current aggregate values and term arguments. The resulting effect is then applied to the target literal's `TargetHeuristicState`.

The native propagator adopts four of clingo's domain heuristic modifiers:

- `level`: sets the branching priority to w (higher values are chosen first), leaving the truth polarity unchanged;
- `sign`: sets the preferred truth polarity based on the arithmetic sign of w (positive for *true*, negative for *false*), using $|w|$ to resolve precedence among conflicting sign directives; since the native language subset admits only strictly positive resolved weights (Section 3.3.3), in this suite the negative branch is never taken;
- `true`: sets the branching priority to w and forces the truth polarity to *true*;
- `false`: sets the branching priority to w and forces the truth polarity to *false*.

clingo's remaining modifiers, `init` and `factor`, are not supported because custom propagators cannot manipulate clasp's internal VSIDS score tables.

If multiple heuristic rules target the same atom, their effects are combined into a `TargetHeuristicState` that tracks the `level` and `sign` across all active `RuntimeHeuristicCandidate` instances.

Unassigned targets with an active `level` are stored in an ordered set:

```
std::set<DecisionRankKey, DecisionRankGreater> active_decision_ranks_;
```

Entries are sorted by decreasing weight. Because weights are stored as standard 32-bit integers, this avoids the 16-bit saturation limit of clasp's internal domain table (Section 3.1.4).

When clasp calls `decide`, the propagator inspects `active_decision_ranks_` from the beginning, discarding any targets that are already assigned on the trail or whose weights have changed. It then branches on the first valid target using its stored `sign`. If the set is empty, the propagator falls back to the atom clasp passes in as its own next decision: when that atom is still unassigned and carries a stored `sign`, the propagator keeps clasp's choice of atom and fixes only its polarity. Otherwise it returns 0 and delegates the decision entirely. During backtracking, `undo` restores aggregates and candidate states through functions declared `noexcept`.

4.2 The Prolog Backend

While the native backend prioritizes evaluation speed through compiled C++ structures, the Prolog backend evaluates heuristics as logic programs via an embedded engine, trading runtime overhead for broader relational expressiveness. This section describes its runtime architecture and trail synchronization mechanisms (with a comprehensive performance comparison presented in Section 5.7).

4.2.1 Rule Strings and Normalization

Section 3.3.2 introduced the notation and the predicates it relies on; recalling the BSP heuristic:

```
heuristic("heuristic(b(X), W, 0, true) :-
    holds(x(X)), target_available(b(X)), alpha_not(c(X)),
    aggregate_all(sum(Y), holds(c(Y)), S), holds(heuristic_base(Base)),
    W is Base * S + X.").
```

During initialization, the propagator collects all `heuristic/1` facts and normalizes the rule strings. In the embedded SWI-Prolog backend, each negative condition explicitly calls either `clingo_not/1` or `alpha_not/1`; the choice is local to that condition, so both predicates can occur in the same body. The benchmark encodings use a consistent choice for each semantic variant. Aggregate readiness is controlled separately by the guards described below. The propagator extracts the predicate signatures referenced in the rules (`holds`, `alpha_not`, `clingo_not`, `target_available`, and rule targets) and tracks only the solver atoms matching those signatures, so that irrelevant predicates cost neither memory nor bookkeeping time.

4.2.2 The Runtime Program

During initialization, the backend starts an embedded SWI-Prolog engine [13] and loads a set of base Prolog rules:

```
holds(A) :- true_atom(A).
holds(A) :- static_atom(A), \+ true_atom(A).
clingo_not(A) :- false_atom(A).
alpha_not(A)  :- \+ holds(A).
target_available(A) :- target_atom(A), \+ true_atom(A), \+ false_atom(A).
dyn_max(Goal, Template, Max) :-
    (aggregate_all(max(Template), Goal, R) -> Max = R ; Max = 0).
```

The second clause of `holds/1` excludes atoms already returned by the first clause. An atom present in both `static_atom/1` and `true_atom/1` would otherwise be enumerated twice on backtracking, potentially duplicating heuristic candidates or contributions to Prolog aggregates. Here, `\+ true_atom(A)` prevents duplicate enumeration; it does not require a static atom to be false in the solver.

Most predicates in these base rules are standard static Prolog definitions, asserted once during initialization and remaining invariant throughout search. The exceptions are `true_atom/1` and `false_atom/1`, which dynamically track the solver's partial assignment. Rather than residing in the Prolog database as dynamic facts, both are implemented as C++ *foreign predicates* via SWI-Prolog's C API. They directly query an in-memory literal store maintained by the propagator: two hash sets (one for true and one for false literals), partitioned by predicate signature. Recording trail changes during `propagate` or `undo` simply updates these native C++ structures, without requiring any calls into the Prolog engine.

Under the clingo-like policy, the native propagator defers an aggregate until every one of its source literals is assigned (Section 4.1.3). The current Prolog backend does not automatically defer `aggregate_all` according to solver assignments: this readiness check is expressed in the encoding using `target_available/1`. Since it succeeds on atoms that are still undecided, `\+ target_available(Pattern)` succeeds only when no atom matching `Pattern` is left undecided. The `lc` encodings carry one such guard per dynamic source of each aggregate; in BSP, where the sum ranges over the `c` atoms, the guard is `\+ target_available(c(_))`. A future translator could insert these guards automatically for supported aggregate forms under this evaluation policy (Section 6.4).

4.2.3 Synchronization and Decision

Trail synchronization is incremental: during search, `propagate` and `undo` record atoms as true, false or unassigned in the store described above, as their solver literals change on the trail.

At each decision point (`decide`), the backend runs the query `heuristic(T, W, P, M)` and collects every solution. A solution is only a proposal: before it can become a decision, its target term must be matched to a solver literal that is still unassigned. The rule can restrict its target using `target_available/1`: at initialization, the backend creates `target_atom/1` facts for known atoms mapped to solver literals,

and `target_available/1` additionally excludes assigned atoms. This implements the domain-filtering role of an auxiliary predicate listing available target terms, and is already used in the benchmark encodings. The final C++ lookup maps the term to a solver literal and retains a defensive check for arbitrary user rules that omit the guard. Binding every variable does not by itself guarantee that the constructed target term occurs in the solver atom table. Among the solutions that survive, the backend branches on the one with the highest priority and weight. Unlike the native C++ backend, which incrementally maintains a sorted set of active targets, the Prolog backend re-evaluates the heuristic query from scratch at every decision step. This design prioritizes relational expressiveness over per-decision efficiency (evaluated in Section 5.8).

4.3 Benchmark Problems and Encodings

4.3.1 Balanced Sum Problem (BSP)

The BSP divides the set of integers $\{1, \ldots, n\}$ into two disjoint sets, `b/1` and `c/1`, such that their sums differ by at most one. It is the motivating example of [9], and the encoding used here as the Alpha reference is the authors' own `Evaluation/Motivating_Example/Balanced_Sum_Alpha.asp`, taken unmodified from the supplementary material of that paper (Section 4.3.4). The heuristic greedily assigns unassigned integers to set b with a weight proportional to the current sum of set c, driving the solver toward balanced partitions. Because all decisions share a single priority level, the current sum S acts as the primary ranking term, with $X \in \{1, \ldots, n\}$, the integer being considered for placement in b or c, resolving ties between equal sums; larger integers receive higher weights.

The exploratory variants introduced in Section 3.1.2 required some specific adjustments on BSP:

- `ga_weak`: The ground heuristic directives omit the negative body guards (`not c(X)` / `not b(X)`), retaining only the static aggregate binding `S = #sum{Y : c(Y)}` without unrolling over candidate values:

```
#heuristic b(X) : x(X), S = #sum{Y : c(Y)}, W = ((n+1)*S)+X. [W, true]
#heuristic c(X) : x(X), S = #sum{Y : b(Y)}, W = ((n+1)*S)+X. [W, true]
```

  Dropping the `sum_value(S)` domain predicate does not reduce the grounding: the equality binding still forces gringo to enumerate every reachable sum, so `ga_weak` materializes the same $\Theta(n^3)$ directives as `gc` and `ga`. What changes is when they fire, since the equality holds only once every atom of the sum is assigned (Section 5.11).

- `la_aux`: Heuristic conditions and aggregate values are pre-materialized:

```
h_b(X,S) :- x(X), not c(X), S = #sum{Y : c(Y)}.
h_c(X,S) :- x(X), not b(X), S = #sum{Y : b(Y)}.
```

The lazy propagator language is then instructed to monitor these auxiliary atoms via the `__body` constructor:

```
__heuristic(__target(b), __body(h_b, __match(0,0), __bind_arg(s,1)),
            __weight(__add(__mul(s, B), self)),
            __semantics(alpha), __modifier(true)) :- B = n + 1.
```

The propagator will track truth-value assignments over the ground `h_b(X,S)` atoms on the trail, testing whether auxiliary trail synchronization reintroduces solver overhead.

- `la_co`: Uses the same lazy heuristic as `la`, but instead of computing and comparing the two sums separately:

```
sumB(Sum) :- Sum = #sum{Y : b(Y)}.
sumC(Sum) :- Sum = #sum{Y : c(Y)}.
:- sumB(SB), sumC(SC), (SB - SC)**2 > 1.
```

it enforces the partition balance directly with a single aggregate rule:

```
#const min_val = tot / 2.
#const max_val = (tot + 1) / 2.
:- not min_val <= #sum{ Y : b(Y) } <= max_val.
```

This removes the heavy grounding bottleneck of the base problem while keeping the meaning of the heuristic unchanged.

4.3.2 Partner Units Problem (PUP)

The PUP [14] assigns communication units to interconnected zones and sensors under capacity and topology constraints. The heuristic uses a two-level priority policy: zones are assigned first, followed by sensors.

The base encoding is taken from the Alpha reference literature [14, 9]: the reference file is the authors' `Evaluation/PUP/Double/encoding-alpha-prologheu-double.asp`, and the `double-`N instances are byte-identical to theirs. Alpha uses the original encoding. Directly grounding that encoding in clingo creates a severe bottleneck, so the clingo variants use a refactored base encoding intended to preserve the same solutions while reducing grounding cost: bounding unit indices to valid ranges, expressing capacity limits via linear `#count` aggregates, and eliminating a redundant blocking rule that generated excessive ground constraints.

Finally, the global priority of zones over sensors is flattened into the heuristic weights using the +100,000 offset introduced in Section 3.1.3.

4.3.3 House Reconfiguration Problem (HRP)

The HRP [15] places items into cabinets and assigns cabinets to rooms under person-ownership and dimensional constraints. The benchmark seeks one valid configuration, with heuristic preferences for reusing the legacy setup rather than an explicit

optimization objective. The heuristic enforces a five-level priority policy: reusing legacy components, placing long items, placing short items, assigning cabinets to rooms, and finally setting unassigned choice atoms to false. It is the heuristic of Case Study 1 of [8]; the reference file is the authors' `DomainHeuristics/House/house_alpha_2021-02-04.asp`, and the encodings of this suite are a rule-by-rule port of it.

Unlike BSP and PUP, the HRP heuristic uses no dynamic aggregates: its conditions are ordinary literals and its weights use arithmetic expressions. The encoding uses aggregates indirectly, in the definitions of `fullCabinet/1` and `fullRoom/1`. HRP therefore isolates the behaviour of lazy default negation (`la` vs. `lc`) on partial assignments.

4.3.4 Alpha as an External Reference

We benchmark our propagator against the lazy-grounding solver Alpha [7, 8] as an external reference. Specifically, we test the *Qh* (Query Heuristics) variant developed by Comploi-Taupe et al. [9], invoked with `-uqh`, which evaluates domain heuristics and dynamic aggregates over partial assignments via Prolog queries. This mechanism directly mirrors our Prolog backend and implements the semantics our native propagator reproduces.

The choice of build matters: the official Alpha 0.7.0 release does not support `#heuristic` directives, and the main upstream branch evaluates aggregates statically. We therefore use the authors' research Qh build, which implements dynamic aggregates over partial assignments.[1] All Alpha encodings and benchmark instances are taken unmodified from that same repository: BSP and PUP from its `Evaluation` directory, the HRP encoding from the test resources of [8] that the same repository also carries.

We evaluate Alpha under two configurations on all benchmark families: `alpha_qh` (with domain heuristics enabled via `-uqh`) and `alpha_noheur` (the baseline solver without heuristics, `-ids`). On HRP, we also run `alpha_dom`, which evaluates domain heuristics natively without the Prolog bridge. In Section 5.9, we compare these runs against our native clingo variants, `la` (lazy propagator), `ga` (ground unrolled simulation), and `gc_noheur` (heuristic-free baseline), to isolate the cost of lazy grounding from the benefit of dynamic heuristic guidance.

Search counters cannot be compared across systems: Alpha's *guesses* and *backtracks* measure lazy CDNL search interleaved with dynamic rule instantiation, whereas clasp records decision *choices* and *conflicts* over a pre-ground program. Cross-system comparisons are therefore limited to external process metrics recorded by `runlim`: wall-clock time and peak resident set size (RSS). Furthermore, because Alpha runs on the Java Virtual Machine (JVM), its RSS includes resident JVM memory as well as the solver's data. The `-Xmx` option sets a maximum heap size, not a measured resident allocation.

[1] `https://github.com/tilmanni/Alpha/tree/domspec_heuristics_extended_prolog`, the build referenced by [9].

4.4 Benchmark Infrastructure and Metrics

The experimental campaigns are orchestrated using `benchmark-tool` [16], the Potassco benchmarking suite designed to automate ASP experiment execution and distribute jobs across SLURM clusters. Each run executes a specific (system, setting, instance) configuration under the supervision of `runlim` [17], which samples resource consumption while enforcing hard wall-clock time and memory limits.

All clingo runs are executed with domain heuristics enabled (`-heuristic=Domain`) and search configured to find a single answer set (`-n 1`). For each run, we record total execution time, solving and grounding components, peak resident memory (RSS), and search statistics including choices and conflicts. Grounding time is estimated as total clingo time minus solving time, clipped at zero; it includes other work outside the solving timer and is approximate on very short runs. We also track the size of the ground heuristic component, comparing materialized directives against lazy facts, along with propagator-specific overheads such as decision calls, query evaluation times, and trail synchronization costs.

Memory is measured as the peak resident set size (RSS) of the entire solver process, which for the Prolog backend includes the embedded SWI-Prolog runtime. Tables and plots retain the benchmark labels MB and GB, with GB values obtained by dividing the recorded MB values by 1024. The configured memory limit is `runlim -space-limit=30000`. Because the engine incurs a constant initialization overhead, lazy Prolog runs can consume more memory than ground variants on small instances; the advantage over the ground-and-solve variants appears as the instances grow.

Ground variants count directives materialized by gringo, whereas lazy variants count template facts. The C++ backend materializes target candidates at initialization; their weights and applicability are updated during search. The runtime effort deferred by lazy grounding is thus captured directly through the per-decision propagator metrics.

Chapter 5

Results

5.1 Experimental Setup and Dataset

All experiments were executed on dedicated HPC cluster nodes requesting 4 cores, with hard limits of 600 seconds and 30 GB of RAM per run enforced by `runlim`. Both backends were compiled on the test hardware from clingo 5.8.0 sources, and the version of SWI-Prolog used was 10.0.2. The campaign was restricted to the fixed node set `kr-node[1-10]` in the SLURM partition `kr`; the archived node log records runs on nine of those nodes. Alpha used the research Qh build from the `domspec_heuristics_extended_prolog` branch described in Section 4.3.4, with a maximum JVM heap of `-Xmx28g`. The archived campaign metadata does not record the processor model or an exact Alpha source revision, which limits exact reproduction of the execution environment.

The chapter first examines heuristic representation and search behaviour on BSP, PUP and HRP, using the native backend for the family plots. Section 5.7 then compares C++ and Prolog directly, and Section 5.8 explains their runtime costs. The remaining sections compare against Alpha, analyze the clingo-semantics variants, and test the ground simulations, auxiliary predicates and compact BSP encoding.

The evaluation covers the benchmark suite defined in Section 4.3:

- BSP: $n \in \{10, 20, \ldots, 200\}$ (20 instances);
- PUP: `double-`N, with $N \in \{20, 40, \ldots, 200\}$ (10 instances);
- HRP: `house-`N, with $N \in \{2, 4, \ldots, 20\}$ persons (10 instances).

Across all benchmark families, every variant that fails at all fails first by timeout (denoted $T@N$ in the summary tables): the 600-second limit is always reached before the 30 GB ceiling, so the frontiers reported here are time-bound throughout. Memory exhaustion does occur, but only on the standard BSP encoding at $n = 180$, where grounding the base program alone exceeds the budget; that is beyond the first failure point of every variant sharing that encoding and therefore never determines one.

On BSP the two backends have identical choice and conflict counts on every instance both solve. On PUP and HRP, `la` choice counts differ by up to about fourteen percent relative to C++; differences can be larger for `lc`. Section 5.7 discusses the effects of candidate ordering and auxiliary predicates on this comparison.

5.1.1 Campaign Overview and Failure Modes

The primary performance comparison is between the heuristic-free baseline and the Alpha-like variants `ga` and `la`, subject to the translation and weight limits discussed below. The `gc` and `lc` variants serve as controls for transferring heuristics designed for Alpha-like partial assignments to a different interpretation; their poor performance does not establish an advantage over correctly adapted clingo heuristics.

Tables 5.1–5.3 summarize the outcome of the benchmark campaign, reporting the number of solved instances and the first failure point for each variant. In the BSP benchmark, solver failures scale predictably with instance size: once a variant fails at a given size, it also fails for all subsequent sizes, with timeouts representing the first failure mode in all cases (between $n = 70$ and $n = 170$). On PUP, however, difficulty does not scale monotonically with size: `lc` times out on `double-80` yet successfully solves `double-100`. With one instance and one run per size, the campaign cannot separate instance-specific difficulty from run-to-run variability. Section 5.5 examines the associated search effort.

	native		**Prolog**	
Variant	**solved**	**first failure**	**solved**	**first failure**
`gc_noheur`	15/20	T@160	15/20	T@160
`gc`	13/20	T@140	13/20	T@140
`ga`	6/20	T@70	6/20	T@70
`ga_weak`	14/20	T@150	14/20	T@150
`la`	**16/20**	**T@170**	**15/20**	**T@160**
`lc`	15/20	T@160	15/20	T@160
`la_aux`	6/20	T@70	8/20	T@90
`gc_noheur_co`	**20/20**	—	**20/20**	—
`gc_co`	**20/20**	—	**20/20**	—
`ga_co`	**20/20**	—	**20/20**	—
`ga_weak_co`	**20/20**	—	**20/20**	—
`la_co`	**20/20**	—	**20/20**	—
`lc_co`	**20/20**	—	**20/20**	—

Table 5.1: BSP campaign overview ($n = 10..200$, 600 s / 30 GB per run). The top block holds the four main variants, the heuristic-free baseline and the `ga_weak` ablation of Section 5.11; the middle block holds the `la_aux` ablation of Section 5.12; the bottom block is the `_co` family (Section 5.13).

	native		Prolog	
Variant	**solved**	**first failure**	**solved**	**first failure**
`gc_noheur`	4/10	T@100	4/10	T@100
`gc`	3/10	T@80	3/10	T@80
`ga`	**10/10**	—	**10/10**	—
`la`	**10/10**	—	**10/10**	—
`lc`	4/10	T@80 (non-mon.)	2/10	T@60

Table 5.2: PUP campaign overview (`double-`N, $N = 20..200$, 600 s / 30 GB per run). `double-200` is the largest instance published with the reference encoding; dashes indicate "solved every available instance".

	native		Prolog	
Variant	**solved**	**first failure**	**solved**	**first failure**
`gc_noheur`	10/10	—	10/10	—
`gc`	10/10	—	10/10	—
`ga`	10/10	—	10/10	—
`la`	10/10	—	10/10	—
`lc`	8/10	T@18	6/10	T@14

Table 5.3: HRP campaign overview (`house-`N, $N = 2..20$ persons, 600 s / 30 GB per run).

The maximum instance size solved provides different insights depending on the problem of choice.

- **On BSP**, the solvability frontier alone does not separate the approaches. Variants sharing the same base encoding (`gc_noheur`, `la`, and `lc`) all stop between $n = 150$ and $n = 160$, constrained by the heavy grounding of the base program. At $n = 120$, `la` takes 0.04 s of solving with zero conflicts, whereas `lc` takes 49.1 s and reproduces the baseline's 78,565 choices and 49,993 conflicts. The ground simulation `ga` also achieves n choices and zero conflicts through $n = 50$, before its measured search deteriorates as discussed in Section 5.11. The compact `_co` comparison in Table 5.5 exposes the grounding difference more directly.
- **On PUP** (Table 5.2), the Alpha-like dynamic guidance lets both `ga` and `la` solve the entire range up to $N = 200$, whereas the other variants stall around $N = 80$ under search space explosion. At $N = 200$, grounding accounts for 98.1% of total clingo time in `ga` and 97.6% in `la`; solving accounts for the remaining 1.9% and 2.4%. The observed advantage is therefore dominated by grounding: `la` approximately halves grounding time and peak memory relative to `ga` (**12.7 GB** vs **24.4 GB**). This is not a comparison with identical rankings, since `ga`'s zone weights saturate (Section 5.5).
- **On HRP**, the frontier is uninformative because nearly all variants solve the entire suite; the evaluation centers instead on decision quality and on the misguidance observed in `lc`.

Three further trends are visible in the tables. First, the static ground simulation (`ga`) fails on BSP already at $n = 70$, nine sizes earlier than the baseline, because saturation of clasp's weight field disrupts the heuristic ranking (Section 5.11). Second, the native C++ and Prolog backends yield identical frontiers whenever propagator queries remain cheap; where they diverge, the Prolog backend usually fails earlier due to evaluation overhead, particularly for `lc`, where the propagator is queried tens of thousands of times. The one exception is the `la_aux` ablation, which reaches two sizes *further* on Prolog, for the reason analyzed in Section 5.8. Third, `lc` shows two distinct failure modes of the clingo-like reading over partial assignments: on BSP the heuristic never activates, because its aggregate conditions are not satisfied at any decision point, whereas on HRP delayed evaluation produces misleading suggestions that degrade search by orders of magnitude (Section 5.10).

5.2 Reduction of the Ground Heuristic Component

The primary benefit of lazy evaluation is eliminating the materialization of heuristic directives during grounding. Figure 5.1 and Table 5.4 compare the number of heuristic entries the grounder materializes across instance sizes on BSP between the ground variants (`gc`, `ga`, `ga_weak`) and the lazy ones (`la` and `lc`, together with the exploratory `la_aux` and `la_co`).

As established in Section 4.3.1, static aggregate unrolling incurs a theoretical $\Theta(n^3)$ combinatorial explosion. The empirical counts reflect this scaling directly: the ground variants scale rapidly from 1,120 directives at $n = 10$ to over 1.34 million at $n = 110$. In contrast, the lazy representation remains invariant at exactly two template facts across all sizes, yielding a reduction factor of over 670,000× at $n = 110$.

n	**Ground directives**	**Template facts**	**Ratio**
10	1,120	2	560×
50	127,600	2	63,800×
100	1,010,200	2	505,100×
110	1,343,320	2	671,660×

Table 5.4: Growth of the ground heuristic component (`gc`, `ga`, `ga_weak`) against the lazily evaluated one (`la`, `lc`). The two columns count different objects, ground `#heuristic` directives against DSL template facts, so the ratio measures how much representation the lazy form never has to build; it is not a speedup.

Crucially, this reduction specifically eliminates the quadratic unrolling over the dynamic aggregate domain (n^2); ground candidate targets (n) continue to be registered at initialization (Section 4.1.1), with candidate evaluation deferred to search.

5.3 BSP: Runtime Behaviour

For all variants sharing the base encoding without grounded heuristic directives (`gc_noheur`, `la`, and `lc`), grounding dominates overall execution time. As shown

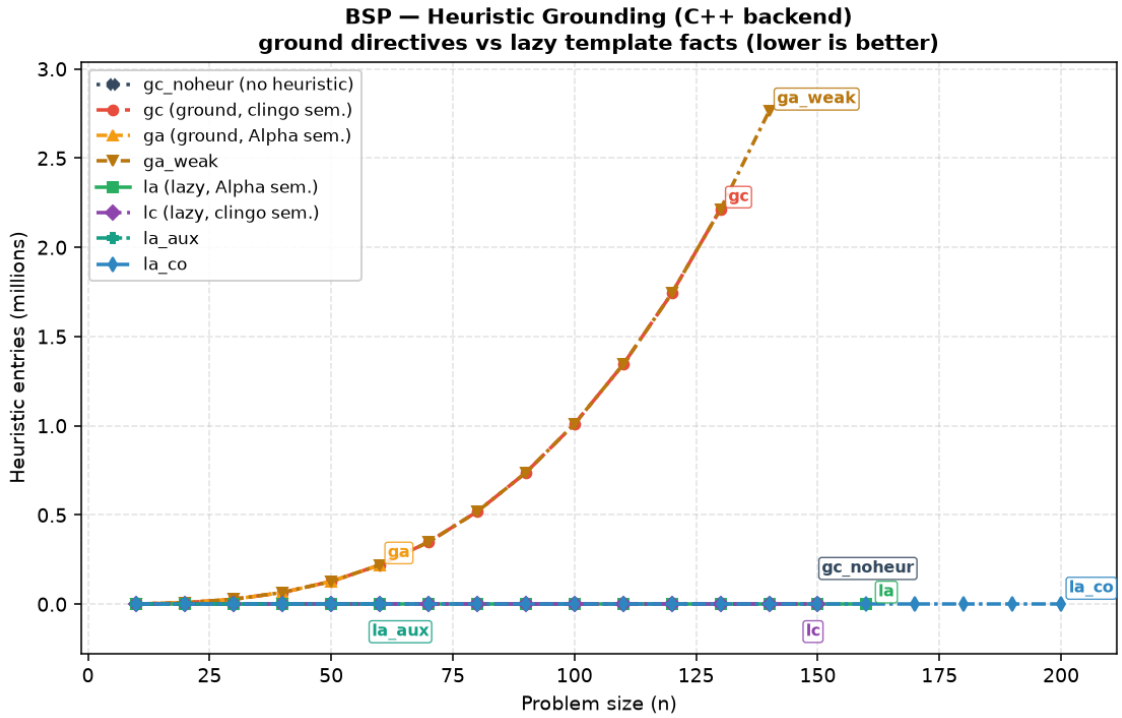


Figure 5.1: Ground heuristic directives against lazy template facts on BSP. The three ground variants coincide by construction, since they unroll the same directive over the same value domain and differ only in the guard that fires it; the four lazy ones and the heuristic-free baseline are indistinguishable from one another at this scale, at two facts and none respectively. Each curve ends at the largest size its variant solves (Table 5.1), so `ga` stops at $n = 60$ although the count itself keeps growing.

in Figure 5.2, their grounding curves remain close across the benchmark range, as none of them materializes heuristic rules during grounding.

The advantage of dynamic evaluation emerges in search performance (Figure 5.3): while `gc_noheur` and `lc` require 44.8 s and 49.1 s of solving at $n = 120$, `la` completes the search in just 0.04 s and remains near zero across all instance sizes.

Although `la` achieves the lowest total runtime among the three variants (Figure 5.4), the overall improvement is approximately 25%. Dynamic evaluation under the Alpha-like reading therefore renders search practically negligible, but overall scaling on standard BSP remains governed by the grounding of the base program, a limitation examined further below.

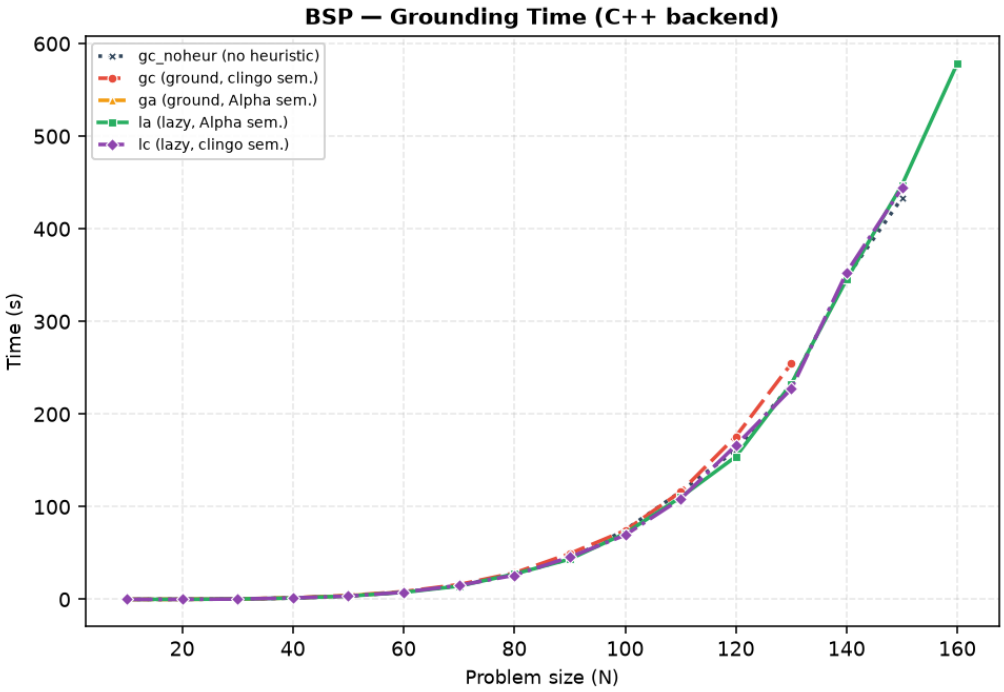


Figure 5.2: Grounding time on BSP for the C++ backend. One run per point; a curve ends where its variant stops solving within the limits.

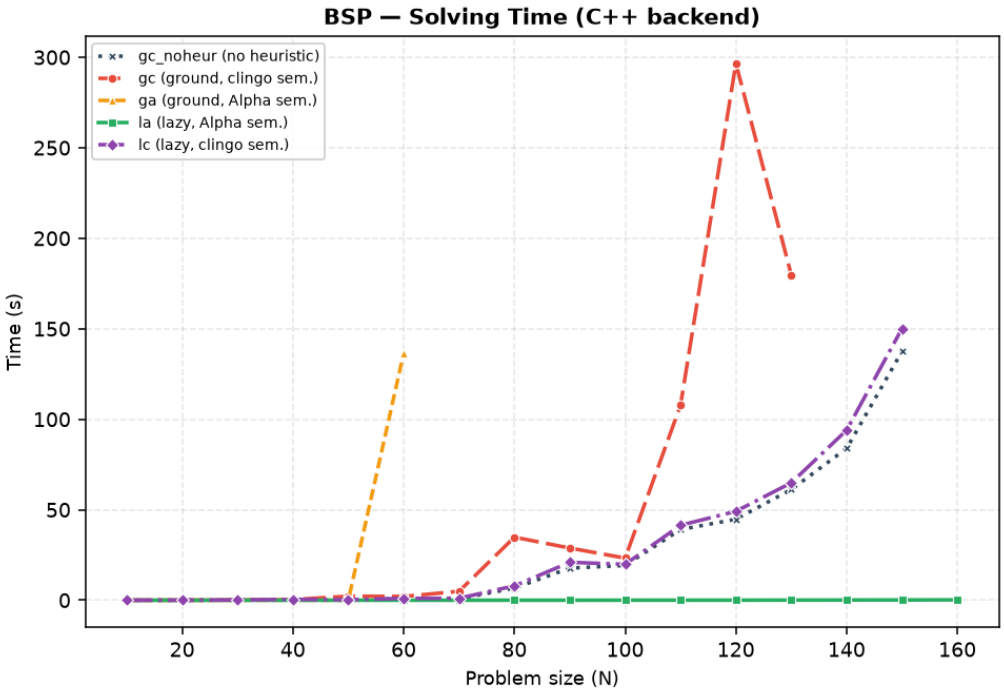


Figure 5.3: Solving time on BSP for the C++ backend, on the same runs as Figure 5.2.

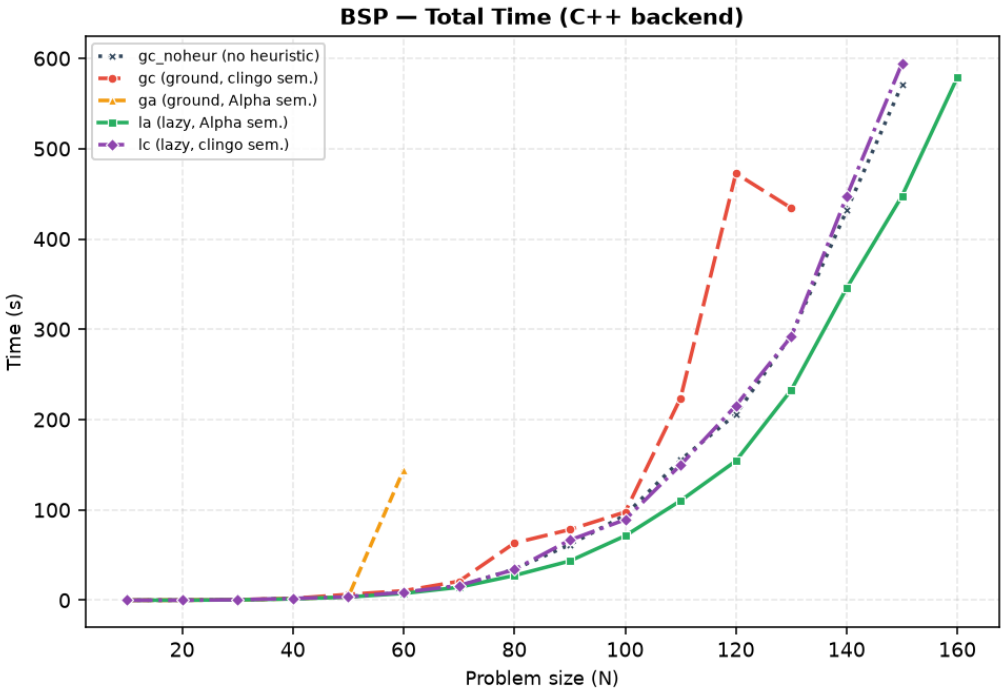


Figure 5.4: BSP total time, grounding plus solving, on the C++ backend.

Variant	Total (s)	Grounding (s)	Solving (s)	Choices	Conflicts
`gc_noheur`	206.0	161.3	44.8	78,565	49,993
`gc`	472.6	176.0	296.6	231,260	160,115
`ga`	T	161.6	438.4	783,034	571,602
`ga_weak`	229.9	163.0	66.9	121,159	80,847
`la`	**154.2**	**154.2**	**0.04**	**120**	**0**
`lc`	215.5	166.3	49.1	78,565	49,993
`gc_noheur_co`	0.016	0.016	< 0.01	37	0
`gc_co`	6.98	6.83	0.15	65	4
`ga_co`	4.16	4.08	0.08	74	3
`ga_weak_co`	3.70	3.63	0.07	74	1
`la_co`	0.006	0.006	< 0.01	117	0
`lc_co`	0.007	0.007	< 0.01	37	0

Table 5.5: Execution time and solver statistics on BSP at $n = 120$. For `ga`, solving time and search counters are recorded at timeout. The two clingo-semantics variants are analyzed in Section 5.10; the `_co` block, which shares the heuristics of the block above and differs only in the formulation of the balance constraint, is analyzed in Section 5.13.

The compact block also provides a direct `la_co`–`ga_co` comparison at $n = 120$: grounding falls from 4.08 s to 0.006 s, about a 680× reduction, while solving takes 0.08 s and less than 0.01 s, respectively. This demonstrates the grounding saving once the large base encoding is removed; it does not demonstrate identical search, and `ga_co` remains subject to weight saturation.

The search statistics in Table 5.5 and Figure 5.5 highlight three key observations:

1. In `la`, default negation and dynamic aggregate evaluation let clingo use the heuristic from the first decision, taking exactly 120 choices with zero conflicts at $n = 120$. Figure 5.5 shows that this is not a property of one size: `la` requires exactly n decisions with zero conflicts at every benchmarked n. Dynamic aggregate evaluation gives the propagator usable guidance from the first decision on an empty assignment, and every choice it proposes lies on the solution path.

2. `ga` and `la` have identical choice and conflict counts up to $n = 50$. At $n = 50$, both take 50 choices with zero conflicts, and solving time rounds to zero at the recorded precision; their grounding times are 3.843 s and 3.574 s, respectively. This provides a comparison before the observed deterioration, while showing how the large base encoding masks the heuristic grounding saving. `ga` then degrades at $n = 60$ and times out from $n = 70$ onward. The cause is clasp's 16-bit weight field, as explained in Section 3.1.4. Section 5.11 analyzes the two ground simulations in detail.

3. The frontier itself, however, is not moved by any of this. `gc_noheur`, `lc` and `la` stop within one size of each other, because around $n = 160$ grounding the base program, already 52.8 million rules at $n = 120$, leaves little or no time for search. On BSP the claim is therefore about cost and not about reach. Section 5.5 exhibits the family where it is about reach; Section 5.13 what happens when the base program is made compact as well; and Section 5.13.1 what becomes of the weight-field limit just described, which turns out to bind the ranking rather than the encoding.

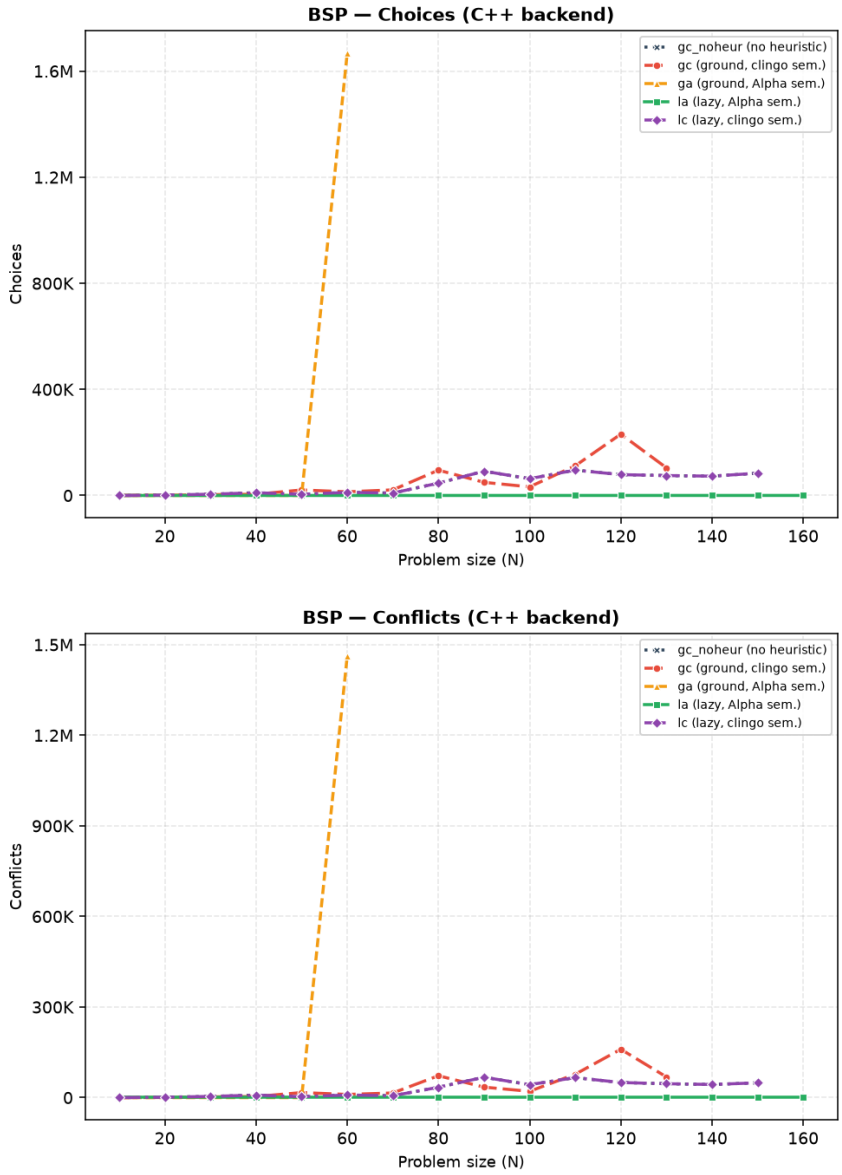


Figure 5.5: BSP search statistics on the C++ backend: solver choices (top) and conflicts (bottom) across sizes.

5.4 BSP: Rules, Variables, and Memory

The reduction of the heuristic footprint is also evident in the size of the propositional formula generated for the solver. As shown in Table 5.6, at $n = 100$ the two lazy variants replace the 1,010,200 ground directives of `gc` with the two template facts of the DSL, and their variable count (20,300) coincides exactly with the heuristic-free baseline: the heuristic leaves no trace in the propositional formula. The ground variants pay this cost in different forms. In `gc` the directives dominate the formula itself, which grows to more than one million variables, fifty times the baseline; in `ga` the variable count stays close to the baseline, but the same million directives still have to be grounded, and beyond $n = 50$ this effort does not translate into usable

guidance. Only `la_co` falls below the baseline, with 101 variables, because it also reformulates the balance constraint (Section 5.13).

Variant	Heuristic entries	Solver Variables
`gc`	1,010,200	1,030,606
`ga`	1,010,200	20,402
`ga_weak`	1,010,200	20,406
`lc`	2	20,300
`la`	2	20,300
`gc_noheur`	0	20,300
`la_co`	2	101

Table 5.6: Heuristic grounding footprint and propositional solver variables at $n = 100$, C++ backend. The first column counts the heuristic entries gringo materializes: ground `#heuristic` directives for `gc`, `ga` and `ga_weak`, the two DSL template facts for the lazy variants, none for `gc_noheur`.

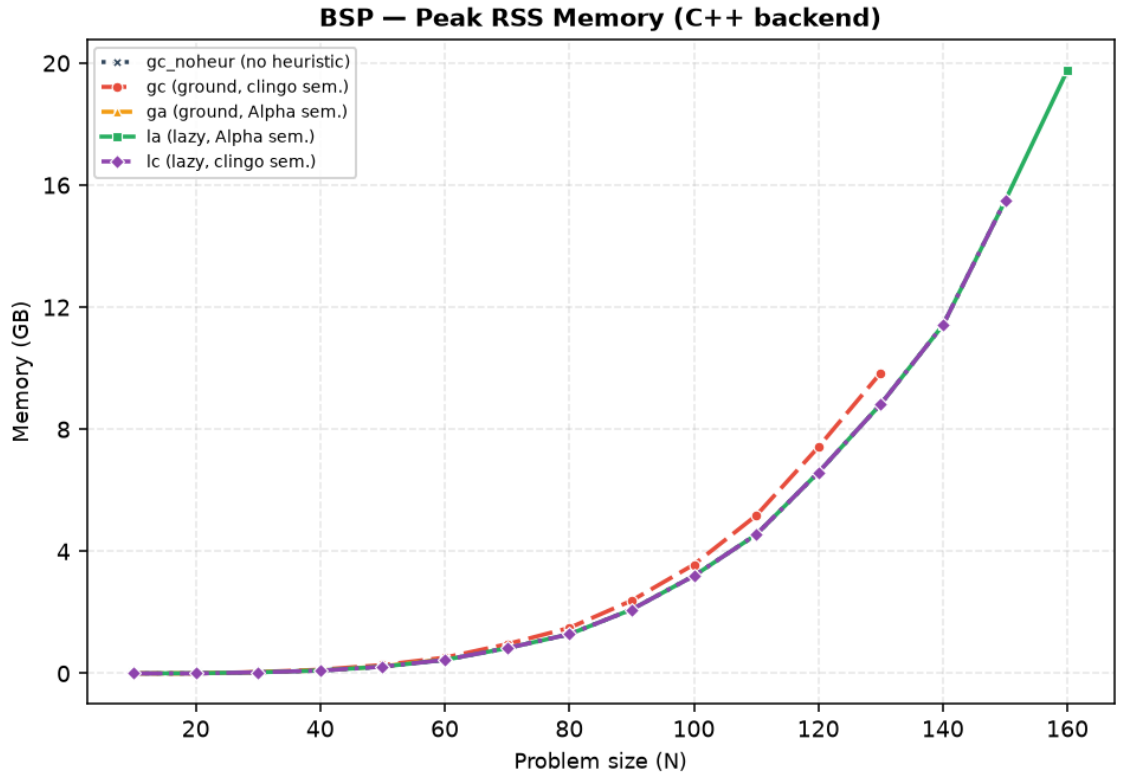


Figure 5.6: BSP: peak resident set size on the C++ backend across instance sizes.

Figure 5.6 shows that on large instances, memory usage is dominated by the base problem encoding; thus, while lazy heuristics eliminate the memory overhead of heuristic directives, scaling on BSP remains bounded by the ground footprint of the base program, a limitation addressed by constraint reformulation in Section 5.13.

5.5 PUP: the Grounding Bottleneck at Scale

Unlike BSP, which uses only `#sum`, PUP combines dynamic `#count` and `#max` aggregates, the latter tracking the remaining capacity across adjacent partner units.

In standard BSP, grounding dominates baseline execution time, and the base program owns the frontier for every variant that shares it. In PUP the dynamic Alpha-like semantics reduces solving effort to a few hundred decisions at every instance size, so more than 95% of the total execution time moves to grounding and the cost becomes one of memory. At $N = 200$, the recorded grounding and solving times are 430.126 s and 8.43 s for `ga`, versus 218.514 s and 5.47 s for `la`: grounding accounts for 98.1% and 97.6% of total clingo time, respectively. The representation cost dominates these measurements, although the two runs do not preserve an identical within-level ranking.

Figure 5.7 and Table 5.7 summarize the overall performance across all five variants under a 600 s timeout and a 30 GB memory limit. The family plots connect successful measurements; the `lc` segment from $N = 60$ to $N = 100$ spans the timeout at $N = 80$.

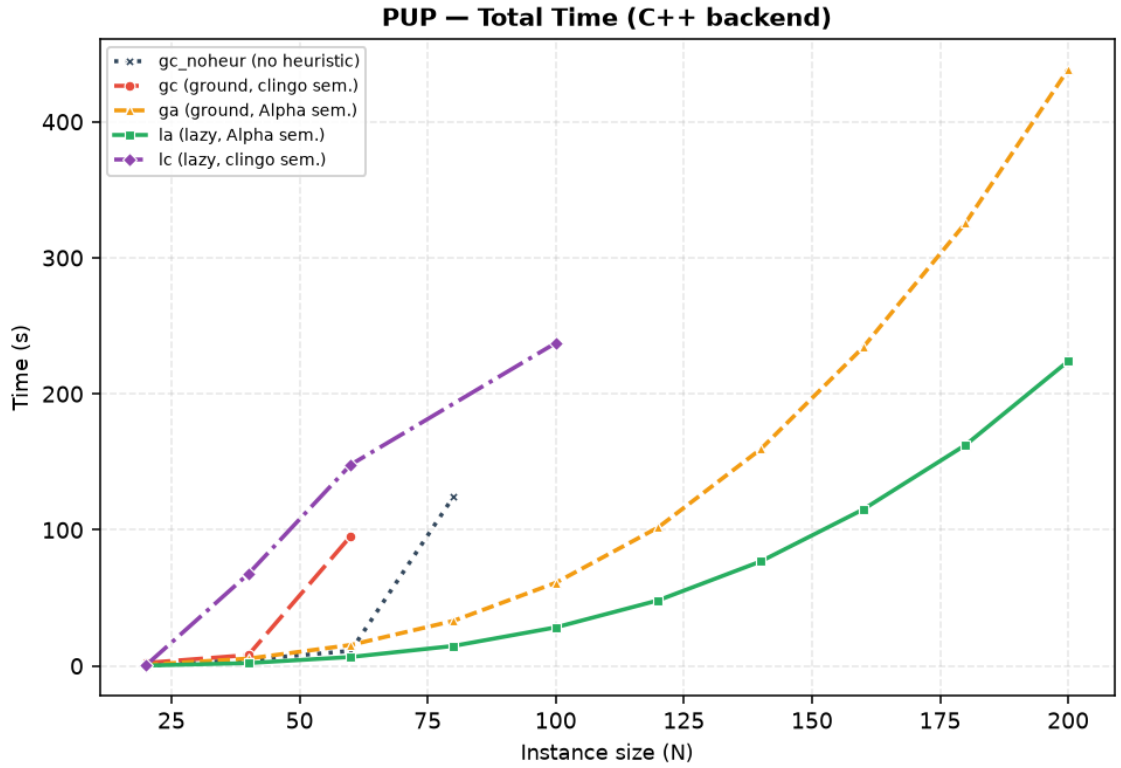


Figure 5.7: Total clingo runtime on PUP across instance sizes for the C++ backend.

Variant	max N solved	Total (s)	Grounding (s)	Peak RSS	First failure
`gc_noheur`	80	124.3	10.7	0.7 GB	T@100
`gc`	60	95.5	17.6	1.3 GB	T@80
`ga`	**200**	438.6	430.1	24.4 GB	—
`la`	**200**	**224.0**	**218.5**	**12.7 GB**	—
`lc`	100	237.4	27.1	2.1 GB	T@80 (non-mon.)

Table 5.7: PUP frontier on the C++ backend (600 s / 30 GB budget): largest solved `double-`N instance and corresponding resource metrics. `double-200` is the largest instance of the reference set.

The RSS values in Table 5.7 refer to different instance sizes: the baseline's 0.7 GB is measured at $N = 80$, whereas `la`'s 12.7 GB is measured at $N = 200$. At the common solved size $N = 80$, the measured peaks are 691 MB for the baseline and 1,175 MB for `la` (about 0.67 and 1.15 GB). The native `la` encoding adds auxiliary rules for heuristic conditions that are absent from the baseline: at $N = 80$, the recorded rule counts are 13,635,973 versus 12,648,338, and solver-variable counts are 1,009,106 versus 93,048. This establishes an additional ground representation cost, alongside the propagator's runtime state. Whole-process RSS does not quantify their separate contributions to peak memory. At $N = 200$, the baseline reaches 9,736 MB before timing out, so this is not a comparison of two completed runs.

The unguided baseline. Without dynamic guidance the solver relies entirely on VSIDS, and combinatorial choices escalate rapidly: `gc_noheur` takes 546,549 choices at $N = 80$ and times out at $N = 100$, where it reaches 3.38 million. The two clingo-semantics variants `gc` and `lc` fail in the same region; Section 5.10 analyzes them together with their BSP and HRP counterparts.

Grounding and memory savings (`la` vs `ga`). Both Alpha-like semantics variants (`ga` and `la`) solve the entire benchmark horizon up to $N = 200$, maintaining near-perfect search guidance (235 choices for `ga` vs 162 for `la` at $N = 120$). With respect to `la`, `ga` has dropped the negative guards and reconstructed the `#max` capacity terms through unrolled bounds (Section 3.1.1), and its zone directives, whose flattened weights start at 100,000, all saturate clasp's 16-bit field (Section 3.1.4). That last effect does not invert the two priority levels, since the largest sensor weight on these instances is 26,220 and stays under the ceiling, so zones are still decided before sensors; what it costs `ga` is the ranking *within* the zone level, which `la` keeps because it ranks its targets internally. This saturation affects every tested PUP size because the zone weights start above the ceiling; selecting only smaller existing instances cannot provide an unsaturated comparison. A comparison with the full intended ranking preserved requires a separately validated encoding or solver change and new measurements. Because solving takes only a few seconds (Figure 5.8), the overall performance is nonetheless differentiated almost exclusively by the grounding footprint.

As shown in Figures 5.9 and 5.10, `ga` must unroll all discrete bounds of the dynamic aggregates into the ground program. At $N = 200$ this costs 430.1 s of grounding and 24.4 GB of memory, against a 30 GB limit. Lazy evaluation in `la`

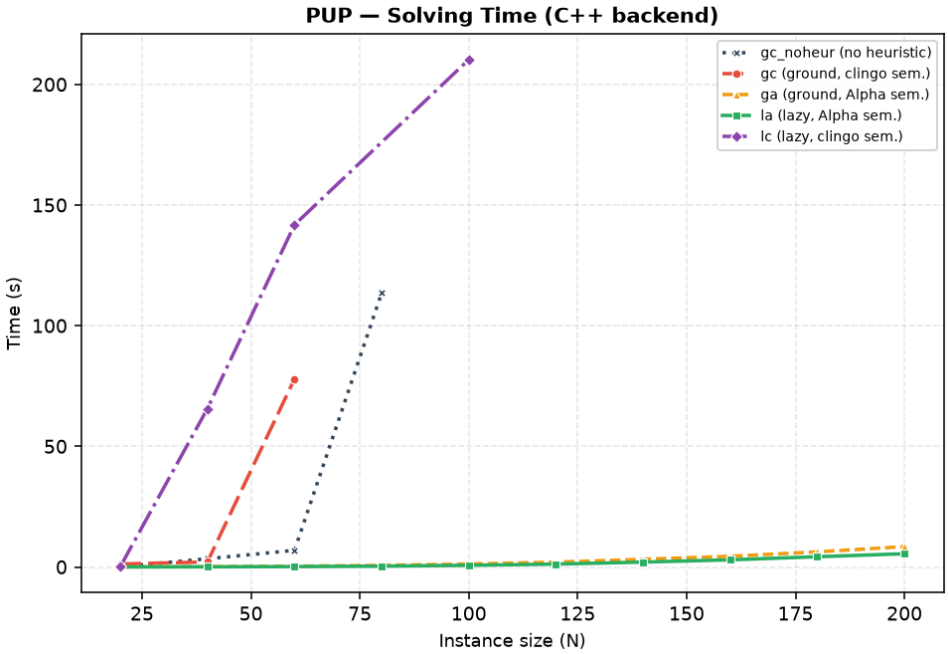


Figure 5.8: Solving time on PUP for the C++ backend.

leaves the aggregates to the in-memory C++ propagator and grounds only the base problem specification, reducing grounding to 218.5 s and peak memory to 12.7 GB. The saving holds across the whole range: the ratio between the two variants is largest on the small instances and settles just below a factor of two at $N = 200$, in both time and memory.

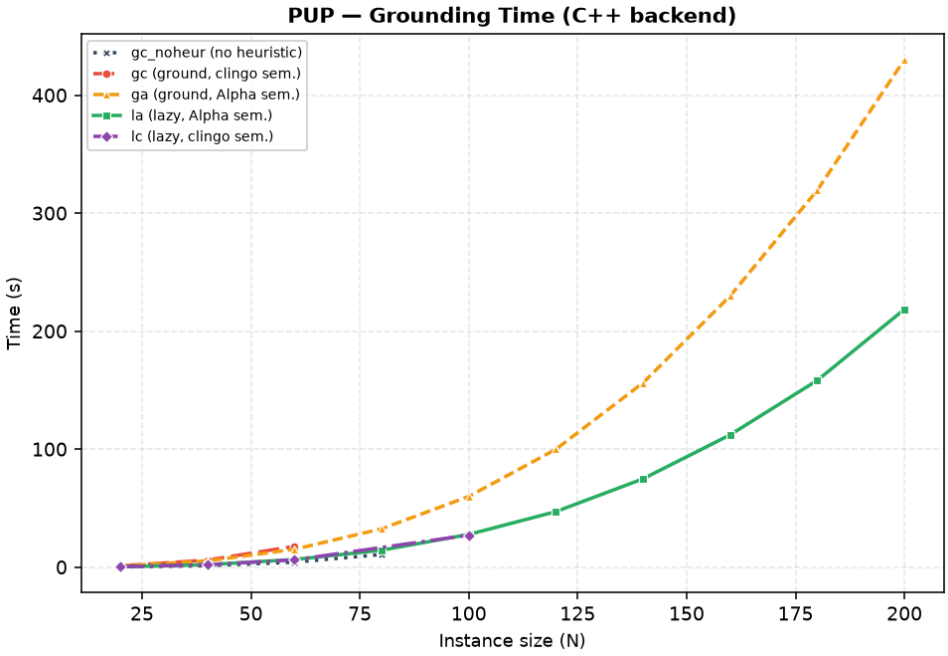


Figure 5.9: Grounding time on PUP for the C++ backend across instance sizes. Note that the vertical scale reaches 430 s against the 210 s of Figure 5.8, on the same runs.

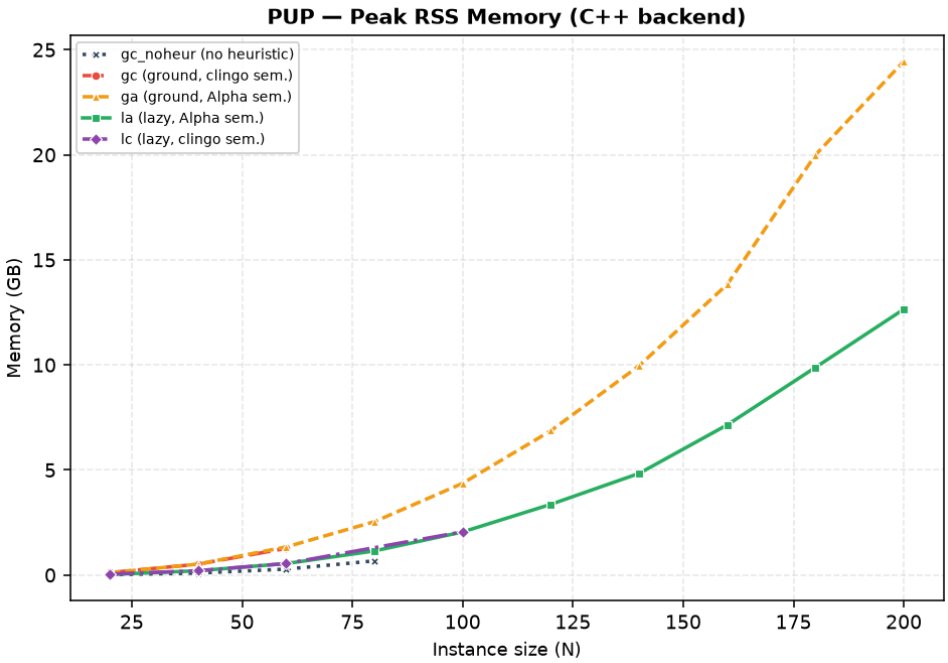


Figure 5.10: Peak memory usage on PUP for the C++ backend, illustrating the 2× memory reduction of `la` over `ga`.

5.6 HRP: Guidance Without Dynamic Aggregates

The HRP comparison includes a semantic limitation: `ga` drops capacity guards that remain relevant to lazy heuristic applicability (Section 3.1.1). Differences between `ga` and `la` therefore cannot be attributed solely to ground versus lazy evaluation of an equivalent heuristic.

HRP evaluates heuristic semantics on rules with nested default negations but without dynamic aggregates. It therefore does not test the aggregate-value expansion studied in Section 5.2. Lazy representation can still avoid many ground directive instances arising from target tuples and body substitutions even without aggregates; the absence of aggregates alone does not establish comparable representation sizes. Because HRP instances are relatively small, nearly all variants solve every instance in under a second on the C++ backend, so the family measures guidance quality (Figure 5.11) and per-decision overhead (Figure 5.13) rather than reach.

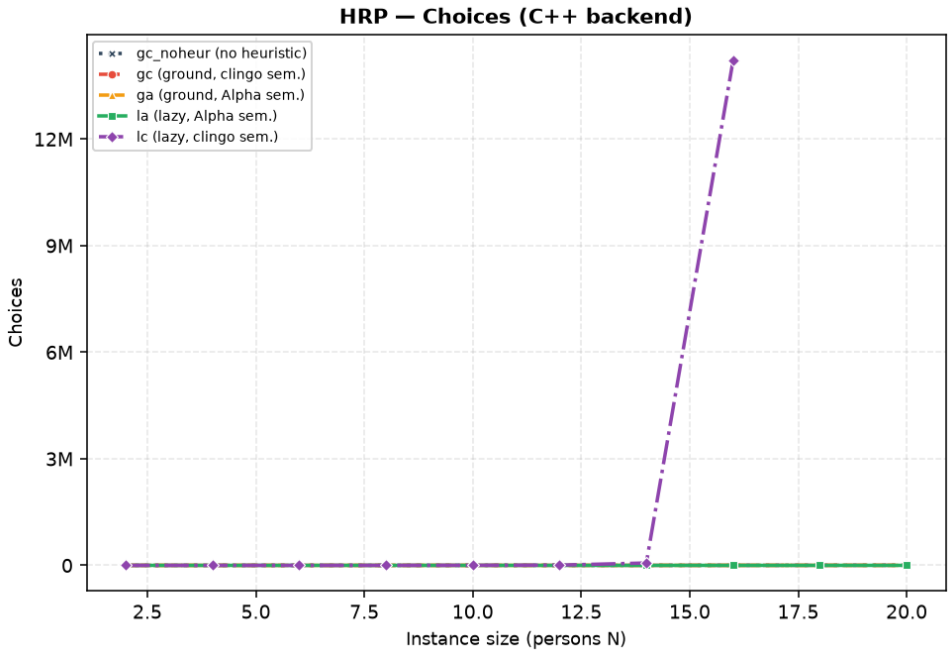


Figure 5.11: HRP: solver choices on the C++ backend. The only diverging curve is `lc`, analyzed in Section 5.10.

Under the Alpha-like reading, `la` and `ga` land within a few choices of each other at every size (86 vs 88 at $N = 14$, 122 vs 116 at $N = 20$, with zero conflicts throughout, Figure 5.11). These counts are consistent with the intended ranking policy, but do not establish semantic equivalence. On this family `ga` is not a reference but an approximation, since its ground form drops the capacity guards `not fullCabinet(C)` and `not assignedThing(T)` rather than reinterpreting them (Section 3.1.1); `la` evaluates those guards on the trail. Section 5.9 compares these results against Alpha itself.

HRP is also the family on which the Prolog backend is most expensive per decision, because the multi-room placement rules join multiple domain predicates

and the engine must enumerate over a thousand candidate pairs at every call (Figure 5.13). That cost belongs to the cost model and is quantified in Section 5.8.

5.7 C++ versus Prolog: Backend Comparison

We now compare both implementations directly. Three quantities need to be distinguished: total runtime, which includes grounding; solving time, where the heuristic evaluator runs; and the search decisions taken by each backend. Table 5.8 reports all three for `la`, together with peak process memory, on representative instances.

Instance	Backend	Total (s)	Solving (s)	RSS (GB)	Choices
BSP $n = 120$	C++	154.21	0.04	6.58	120
	Prolog	163.06	0.10	6.58	120
PUP $N = 200$	C++	223.98	5.47	12.65	272
	Prolog	235.07	8.18	12.55	249
HRP $N = 20$	C++	0.41	0.01	0.049	122
	Prolog	8.08	7.64	0.051	106

Table 5.8: Direct backend comparison for `la`. Times are clingo's internal measurements; RSS is the external process measure. Each row is one run.

Figure 5.12 extends the comparison across sizes. On BSP, both backends take exactly n choices with zero conflicts on the standard encoding, and search remains negligible next to grounding. On PUP, Prolog increases solving time from 5.47 to 8.18 s at $N = 200$, while total runtime rises only from 223.98 to 235.07 s. Most of both runs is spent grounding, so the difference between evaluators is much smaller in the total-time panel than in the solving-time panel. Their peak memory is also similar at this size: both retain the reduction relative to the ground `ga` variant, which uses 24.4 GB.

HRP shows the opposite balance. Its ground program is small, so repeated Prolog evaluation is visible in total runtime: 8.08 s against 0.41 s on `house-20`. Both backends find a solution without conflicts, but the Prolog engine spends 7.64 s in search. This makes HRP the clearest example of the cost of relational evaluation even when the heuristic provides effective guidance. Section 5.8 breaks that cost down into query evaluation, candidate handling and trail maintenance.

The comparison includes differences in both representation and search. On PUP and HRP, the native encoding uses auxiliary predicates for conditions that Prolog evaluates directly. These predicates express joins and relational conditions outside the native template's supported fragment (Section 3.3.3); Prolog can express those conditions directly as goals. For example, `house-20` has 64,634 solver variables in C++ and 53,964 in Prolog. Equal-rank candidates may also be selected in a different order: on `double-200`, C++ takes 272 choices and 26 conflicts, while Prolog takes 249 choices and 54 conflicts. The runtimes therefore compare the complete implementations, including these differences.

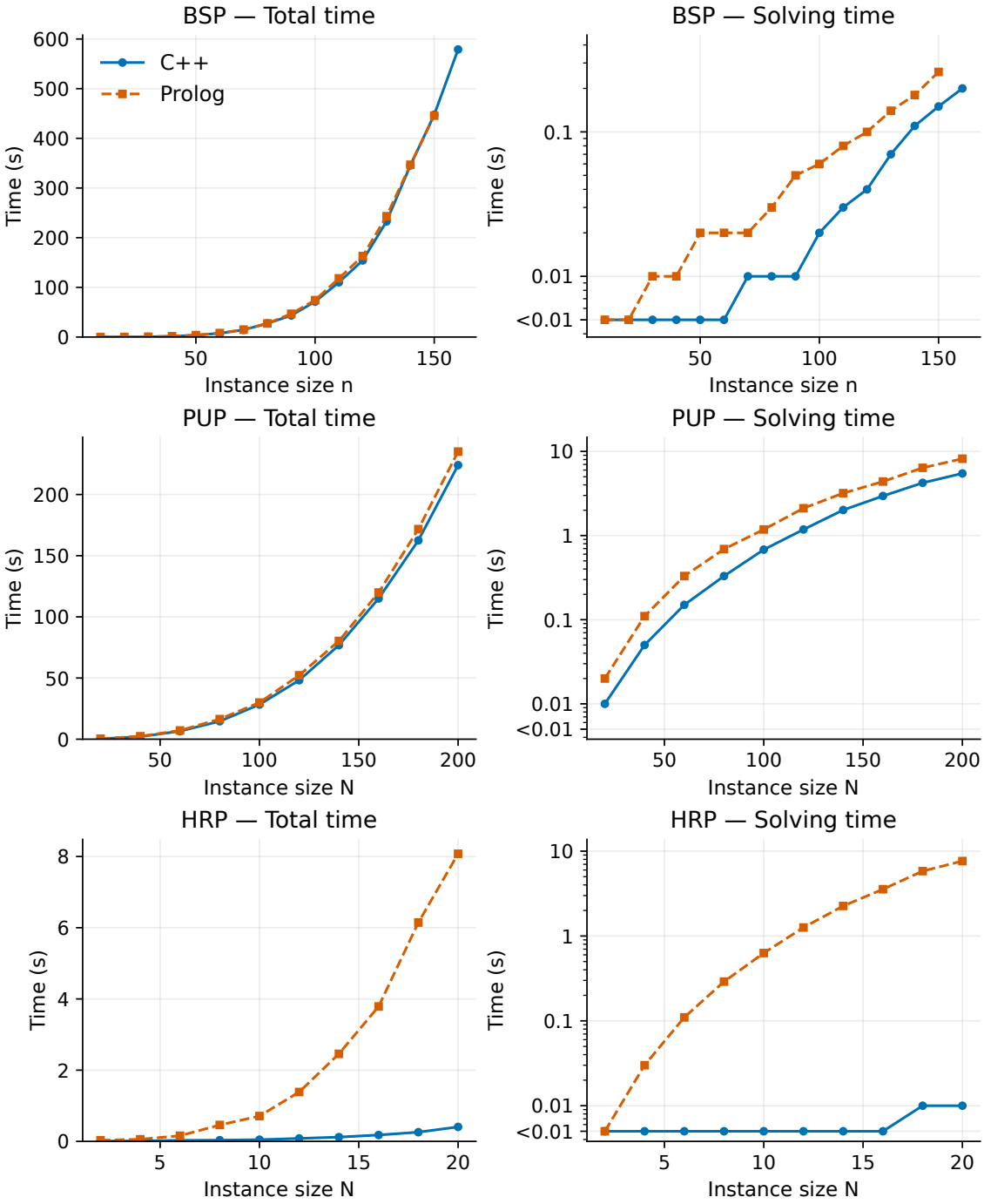


Figure 5.12: The `1a` variant on C++ and Prolog across all three families: total clingo time (left) and solving time (right). Each panel has its own vertical scale; the solving panels use a logarithmic scale to keep both backends visible. Only successful runs are plotted, and curves end at the largest solved instance.

When evaluation cost changes the solved set. The campaign overviews (Tables 5.1–5.3) show four configurations with different solved sets. Prolog solves one fewer instance for `la` on BSP (15 against 16), two fewer for `lc` on PUP (2 against 4), and two fewer for `lc` on HRP (6 against 8). The BSP difference lies close to the time limit and needs repeated measurements to establish its stability. On PUP and HRP, `lc` combines ineffective or misleading guidance with repeated Prolog queries. Conversely, `la_aux` on BSP solves two more instances with Prolog (8 against 6); Section 5.12 discusses this exploratory encoding separately.

Figure 5.13 extends the solving comparison to both lazy semantics and to the ground control `gc`. The control uses clasp's native directives in either executable, so it does not invoke the lazy query engine. The logarithmic scale keeps `la` visible alongside the much more expensive `lc` runs. The overview tables report the failed runs omitted from the curves.

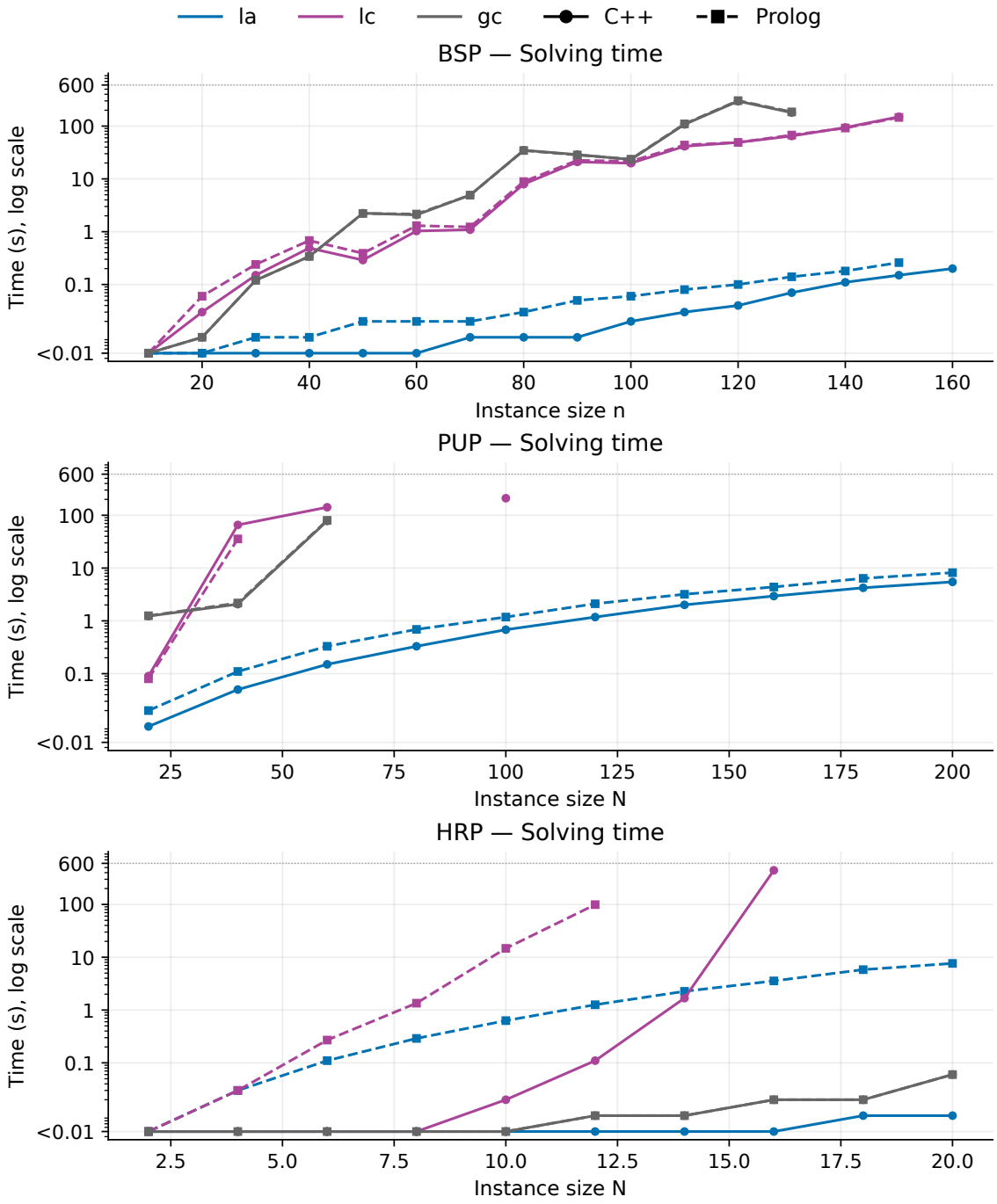


Figure 5.13: Solving time on both backends for `la`, `lc` and the ground control `gc`. Colour identifies the variant; solid circles denote C++ and dashed squares Prolog. Only successful runs are plotted. The dotted line marks 600 s for orientation, although the actual budget includes grounding as well. Values reported below 0.01 s are shown at a plotting floor.

5.8 The Price of Laziness: Per-Decision Cost

The lazy evaluation model represents a space–time trade-off: instead of a combinatorial cost in memory during the grounding phase, there is a cost in computation incurred during search, through state updates and decision callbacks. Now that all three families have been presented, that cost can be reported for all of them at once. There is also a remarkable difference between the two backends: the C++ one is orders of magnitude faster in its decisions.

		Decide calls		**Cost (ms/decide)**		**Prolog backend only**		
Family	**Var.**	**C++**	**Prolog**	**C++**	**Prolog**	**cand./call**	**sync (s)**	**prop. share**
BSP n=120	`la`	120	120	$0.9{\cdot}10^{-4}$	0.45	121	1.0 ms	0.03%
BSP n=120	`lc`	78,565	78,565	$1.5{\cdot}10^{-4}$	0.07	0.0	0.29 s	2.6%
PUP N=160	`la`	217	197	$4.4{\cdot}10^{-4}$	9.73	5.3	0.37 s	1.9%
PUP N=200	`la`	272	249	$5.2{\cdot}10^{-4}$	13.09	5.2	0.63 s	1.7%
HRP N=20	`la`	122	106	$1.5{\cdot}10^{-4}$	71.93	1,248	3.7 ms	94.5%
HRP N=14	`lc`	66,717	T	$0.9{\cdot}10^{-4}$	—	—	—	—

Table 5.9: Per-decision overhead on representative solved instances of all three families. The last three columns report Prolog measurements; propagator share is the sum of decision and synchronization time divided by total clingo time. The `lc` row on HRP has no Prolog data because that configuration exhausts the time limit.

The HRP figures motivate further Prolog optimization rather than establish an unavoidable language cost; possible optimizations are discussed as future work in Section 6.4.

The overall overhead varies dramatically across the benchmark configurations, and Table 5.9 is ordered to make the four regimes visible:

- When the heuristic has **perfect guidance (`la` on BSP)**, with only 120 decide calls taking 0.45 ms each in Prolog (and 10^{-4} ms in C++), propagator overhead accounts for less than 0.05% of the total runtime.
- When the semantics gets in the way **(`lc` on BSP)**, each single query is very cheap (0.07 ms) because it finds nothing to return, but the calls are made 78,565 times instead of 120: the propagator ends up costing 2.6% of the total runtime, roughly eighty times what it costs in `la`, and none of that work helps the search.
- When candidate generation becomes heavy **(`la` on HRP)**, producing over 1,200 candidates per decision, Prolog evaluation time rises to 71.9 ms per call; as a result, the propagator accounts for 94% of the total runtime (7.6 s out of 8.1 s, against 0.41 s of total clingo runtime on the compiled backend), even though search needs only about 100 decisions.
- With frequent decisions and costly evaluation **(`lc` on HRP)**, the accumulated query cost reaches the time limit: a run that the native backend clears with 66,717 choices in 1.80 s exhausts the 600 s budget on the Prolog one. This is one of the three configurations where Prolog solves fewer instances than C++ (Section 5.7).

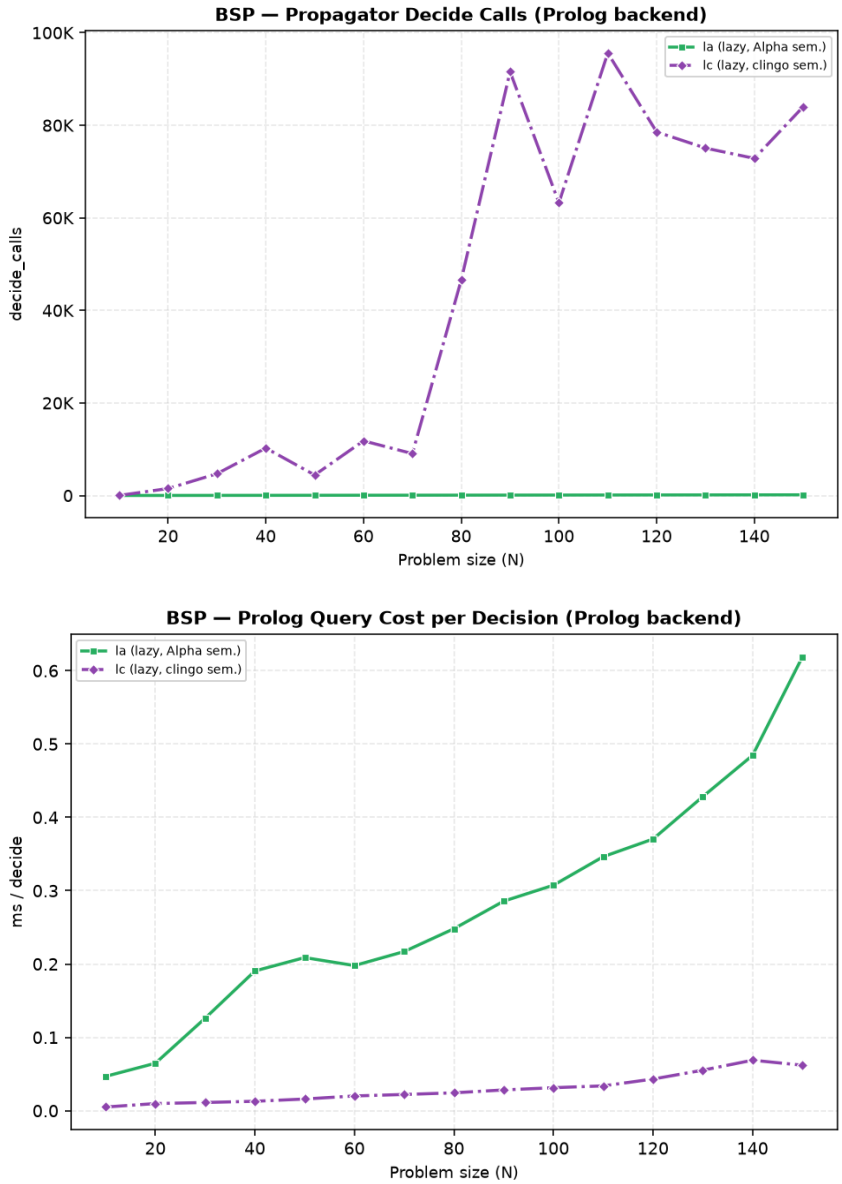


Figure 5.14: BSP, Prolog backend: decide calls (top) and per-decision query cost (bottom). The `lc` curve pairs tens of thousands of calls with a low unit cost; `la` the opposite.

Figure 5.14 separates the two components that determine the Prolog backend's runtime: the number of decision calls and the evaluation cost per call. The lower plot isolates the inference time spent inside the Prolog query itself, which constitutes the vast majority of the per-decision cost (detailed in Table 5.9). At $n = 120$, for instance, the Prolog query accounts for 0.37 ms out of the total 0.45 ms per decision in `la`, and 0.043 ms out of 0.067 ms in `lc`, with the remainder spent on candidate matching and literal lookups on the C++ side.

- **Under Alpha semantics (`la`):** The query actively identifies all eligible candidates on the partial trail, performing term unification and dynamic weight computation. Although this makes each individual decision more expensive (0.37 ms),

the guidance is accurate, keeping the total number of decision calls to a minimum.

- **Under clingo semantics (`lc`):** Negative body literals fail immediately on partial assignments, so the query aborts after finding zero candidates. While this allows each call to terminate roughly eight times faster (0.043 ms), it provides no guidance whatsoever to the solver, triggering a combinatorial explosion in the number of decision calls (upper plot).

Where the Prolog cost actually sits. The profiling data above reflects an architectural redesign of the Prolog backend. In our initial implementation, state synchronization between clingo and Prolog was a severe bottleneck. Every change on the solver trail was mirrored into Prolog's dynamic database by formatting the assignment as a text string and parsing it with `PL_chars_to_term`. This serialization overhead dominated execution: on the PUP benchmark at $N = 200$ (`double-200`), trail synchronization consumed 17.5 s, compared to just 1.75 s of actual query evaluation.

To eliminate this bottleneck, the current design avoids mirroring the trail into Prolog altogether (Section 4.2.3). Instead, the solver exposes its assignment directly to Prolog through non-deterministic *foreign predicates* (`true_atom/1` and `false_atom/1`). Recording a trail assignment is therefore reduced to a native C++ hash-map update, with zero Prolog overhead.

The impact of this redesign is shown in Figure 5.15 and Table 5.9. On `double-200`, synchronization time drops from 17.5 s to 0.63 s, a 27× speedup that brings it on par with the native C++ backend (0.68 s). As a result, the primary computational cost shifts from state synchronization to actual heuristic inference (3.25 s of query time versus 0.63 s of synchronization).

However, accessing the trail via foreign predicates introduces a new trade-off that depends on argument binding:

- **Ground arguments (lookup):** When arguments are known constants (as in the membership tests of `alpha_not/1` and `target_available/1`), the predicate performs an immediate $O(1)$ hash-map lookup on the C++ side.
- **Non-ground arguments (generation):** When an argument contains unbound variables, the predicate must act as a generator. In the current implementation, it does so by copying all matching atoms into a temporary vector before iterating over them.

Because of this vector copy, heuristic rules that perform heavy relational joins and candidate enumeration incur an overhead that the previous dynamic-database design avoided, since Prolog could index clauses directly in its internal database without intermediate copying.

The HRP benchmark clearly illustrates this drawback. Its placement rules join multiple domain predicates, producing 1,248 candidate bindings per decision. Under the new foreign-predicate design, the per-decision evaluation cost rises from 9.19 ms to 71.9 ms. This slowdown turns `house-20` from a 1.5 s run into an 8.1 s one, and pushes `lc` on `house-14` past the time limit.

The transition to foreign predicates reduces synchronization time on PUP. Separately, on BSP, the `la_aux` variant of Section 5.12 solves five additional instances

relative to the earlier Prolog implementation, extending its last solved size from $n = 30$ to $n = 80$. However, it introduces a performance penalty on encodings that require heavy candidate enumeration. Section 6.4 outlines how direct indexed iteration without vector copying can resolve this remaining limitation.

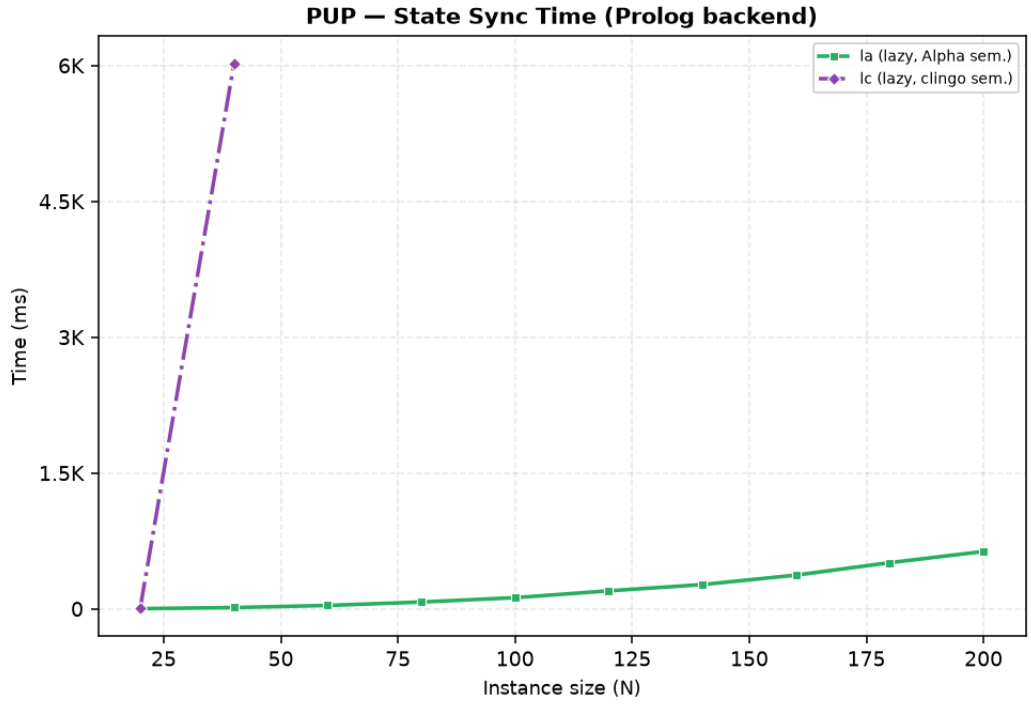


Figure 5.15: Cumulative state synchronization time for the Prolog backend on PUP, with `true_atom/1` and `false_atom/1` as foreign predicates. For `la` it grows smoothly to 0.63 s at $N = 200$, below the 3.25 s the same run spends inside the Prolog query; the `lc` curve, which pays one update per literal on an unguided search, reaches 6 s already at $N = 40$ and stops there.

The PUP comparison in Figure 5.13 shows that solving time for `la` on PUP stays under ten seconds across the whole horizon on both backends, with the Prolog engine holding a small and stable factor over the native one. Both backends deliver guidance of broadly similar quality here, keeping the search down to a few hundred choices on either engine.

The total lazy overhead therefore factorizes as:

$$\text{Total Overhead} = \text{Decide Calls} \times (\text{Query Cost} + \text{Amortized Synchronization Cost}),$$

where Query Cost includes the decision callback and candidate handling, and synchronization is amortized over the decision calls. Figure 5.13 reports overall solving time, which also includes clasp's own search work. Effective guidance reduces the number of consultations: a richer condition costs more per decision, but a heuristic that guides well leaves few decisions to pay for.

5.9 An External Reference: Alpha

To evaluate how closely our propagator reproduces the lazy evaluation process and performance advantages, we compare our clingo variants against Alpha with and without its declarative heuristic engine (Qh mode), on all three benchmark families. BSP and PUP carry the dynamic aggregates the mechanism is designed for and are treated first; HRP, whose reference heuristic has none, is covered separately below because it answers a different question.

On BSP, the guided Alpha configuration (`alpha_qh`) solves all twenty instances in 4.2 s and 0.38 GB at $n = 200$, requiring exactly n decisions and zero backtracks. The `la` variant in clingo likewise takes n choices with zero conflicts, although the counters alone do not establish identical decision sequences. The performance gap at larger sizes (579.7 s for `la` at $n = 160$ versus 2.3 s for `alpha_qh`, both on the `runlim` clock) is dominated by base program grounding in gringo (52.8 million rules at $n = 120$), whereas Alpha avoids materialising the full ground program before search by instantiating rules lazily along the search trail.

Lazy grounding alone is not sufficient without dynamic heuristic guidance: as Figure 5.16 shows, `alpha_noheur` solves nothing beyond $n = 40$, eleven sizes short of clingo's ground baseline `gc_noheur` ($n = 150$). The two systems sit on opposite sides of one trade-off:

- **clingo:** Because the program is fully grounded before search begins, exploring unpromising search branches is computationally very cheap: clasp only has to perform fast boolean propagation and VSIDS variable scoring over pre-existing clauses in compiled C++.
- **Alpha:** Because rules are grounded on demand during search, exploring unpromising branches can require additional rule instantiation and nogood generation.

As shown in Figure 5.17, `alpha_noheur` suffers a severe search space explosion, peaking at over 67,000 guesses and backtracks at $n = 30$; it still solves $n = 40$, where each guess costs more even though fewer are needed, and from $n = 50$ it no longer finishes at all. With heuristic guidance, `alpha_qh` never backtracks on these instances and grounds rules along a successful search path. On this benchmark, dynamic guidance and lazy grounding therefore complement each other.

Partner Units Problem (PUP). On PUP, clingo with lazy heuristics (`la`) closely approaches Alpha's performance, solving $N = 200$ in 227.1 s and 12.65 GB compared to 50.9 s and 12.82 GB for `alpha_qh`: a factor of 4.5 in time, and a peak memory that is marginally *below* the reference. The memory figures should not be read as a win, however, and the reason is stated in Section 4.3.4: RSS includes the resident JVM and Prolog runtimes as well as solver data, and one run per configuration does not establish the significance of a ratio of 0.99. The measured peaks are similar; no memory advantage is established at this size. In contrast, the static ground simulation `ga` required 444.8 s and 24.45 GB, nearly nine times the runtime and 1.9 times the memory of `alpha_qh`. Furthermore, `alpha_noheur` solves no instances on PUP, reaffirming that dynamic heuristics are essential for Alpha on this

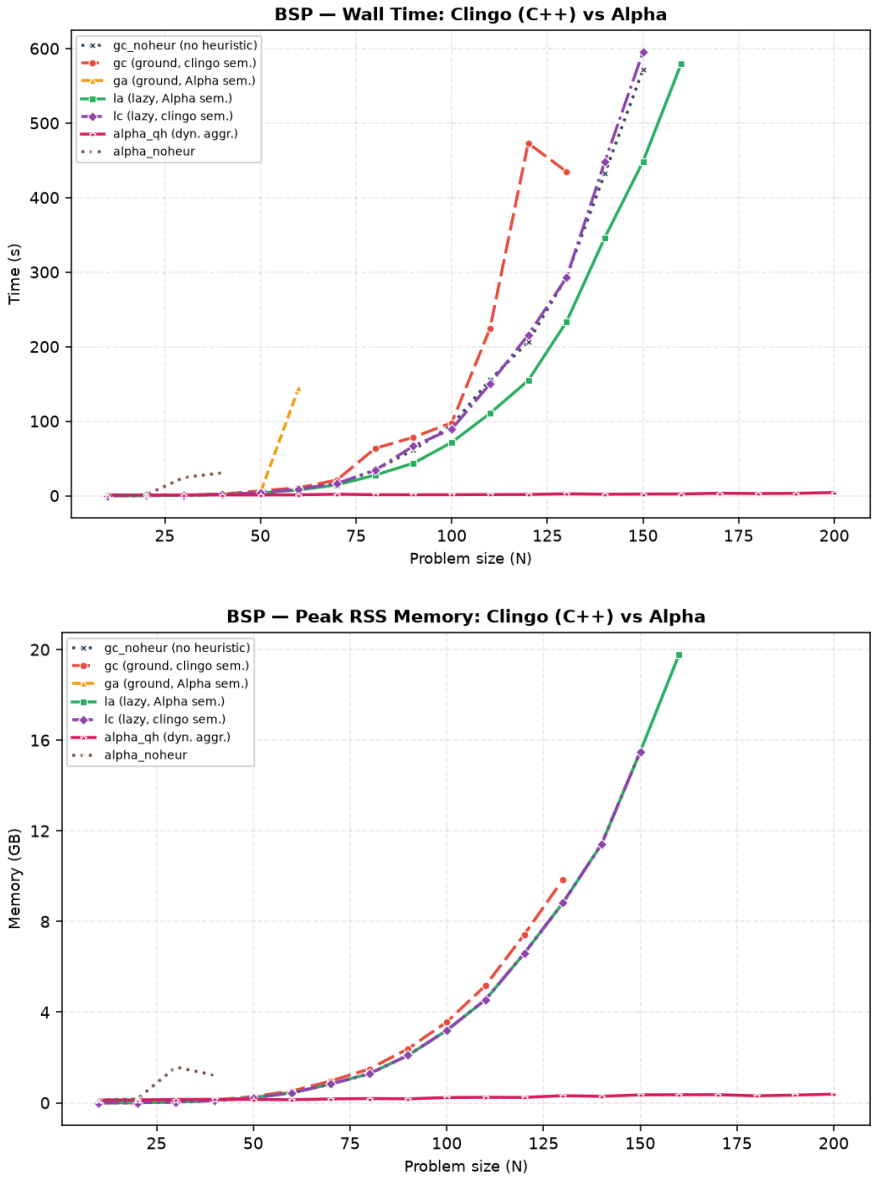


Figure 5.16: BSP benchmark dashboard: the four clingo variants and the heuristic-free baseline `gc_noheur` compared against Alpha with (`alpha_qh`) and without (`alpha_noheur`) declarative heuristics. Top: total wall-clock time; bottom: peak RSS. Both are `runlim` external process measures.

PUP suite within the tested budget. Figure 5.18 places the three systems on the same time and memory axes.

House Reconfiguration Problem (HRP). HRP is the one family whose reference heuristic contains no dynamic aggregates, so it does not test savings from unrolling aggregate values; it does, however, settle the question left open in Section 5.6, namely what `la` should be compared against once `ga` is known to be an approximation. Alpha is run here in all three configurations, including `alpha_dom`, which reads the same domain heuristic natively instead of through the Prolog bridge

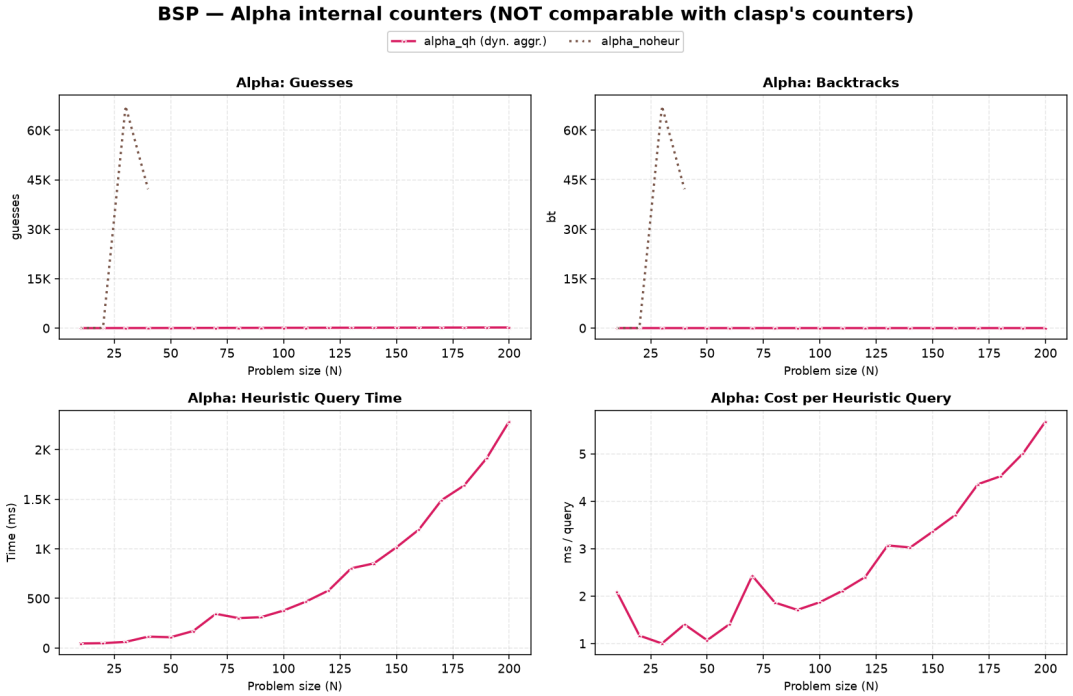


Figure 5.17: BSP internal search counters for Alpha across instance sizes. Top row: with heuristics (`alpha_qh`), guesses remain equal to n with zero backtracks; without heuristics (`alpha_noheur`), search explodes past $n = 20$ and the curve stops at $n = 40$, the last size it solves. Bottom row: cumulative Prolog query evaluation time for `alpha_qh`, which reaches 2.3 s at $n = 200$ and never costs more than 5.7 ms per consultation.

(Section 4.3.4). Table 5.10 reports the largest instance, and Figure 5.19 the whole family.

Configuration	solved	Wall (s)	Peak RSS	guesses / backtracks
`alpha_noheur`	3/10	*T*	—	—
`alpha_qh`	10/10	15.33	1.70 GB	2,368 / 2,142
`alpha_dom`	10/10	1.79	0.42 GB	2,314 / 0
clingo `la`	10/10	0.51	50 MB	122 / 0 *(choices / conflicts)*

Table 5.10: HRP at `house-20`, the largest instance. Alpha counters are *guesses* and *backtracks* over a lazily instantiated program and are not commensurable with clasp's choices and conflicts (Section 4.3.4); they are reported to compare Alpha's three configurations with one another. `alpha_noheur` solves only `house-2`, `house-6` and `house-8`, the last in 106.0 s with 26,086 guesses; the failure at `house-4` shows that success is not monotone in instance size; this single-run campaign does not establish its cause.

Three things follow. First, the heuristic matters here as much as on the other two families: without it Alpha solves three instances out of ten and needs 106 s

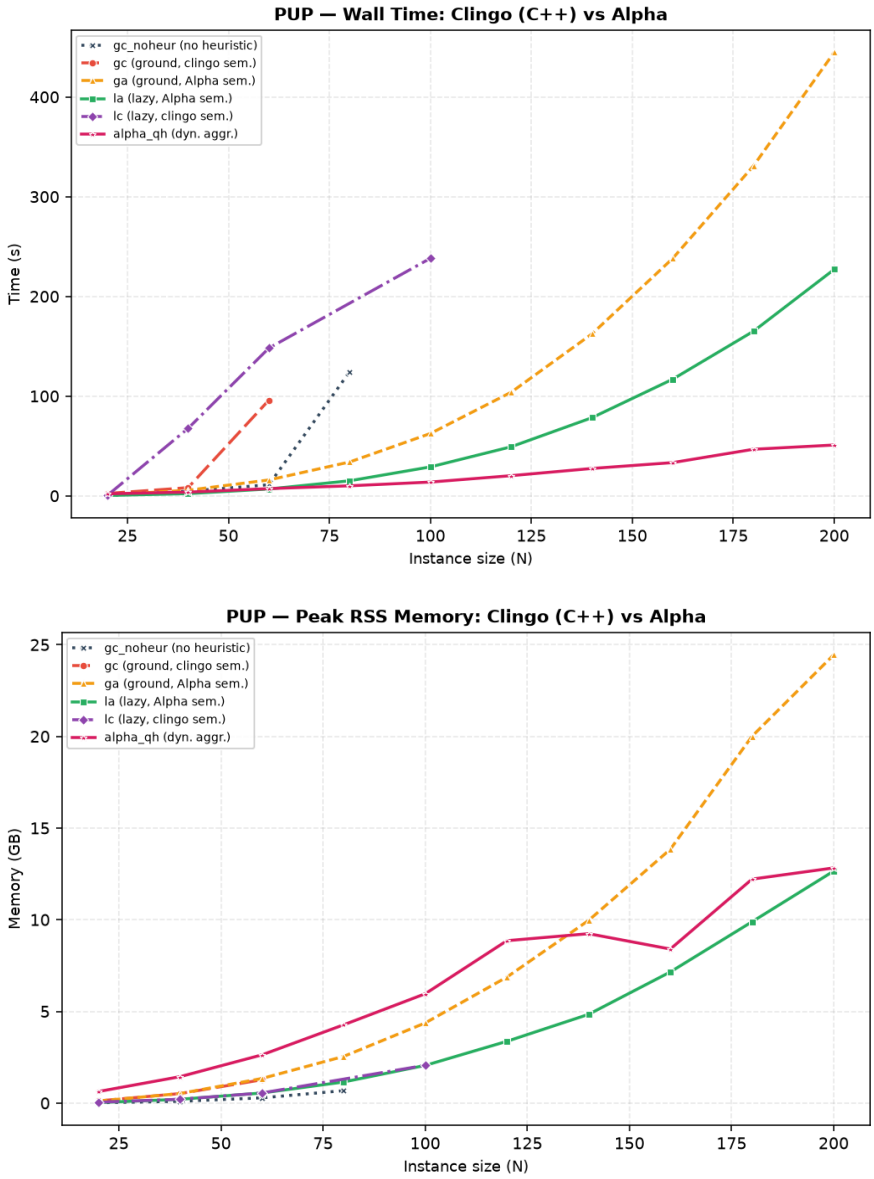


Figure 5.18: PUP benchmark dashboard: external comparison between clingo variants and Alpha. Top: total wall-clock time; bottom: peak RSS. The baseline `alpha_noheur` does not appear as it solves zero instances; the `lc` line connects successful runs across the timeout at $N = 80$.

for `house-8`, while both guided configurations clear the suite in seconds. Second, and unlike BSP and PUP, clingo is not the slower system on this family: `la` solves `house-20` in 0.51 s and 50 MB against 15.33 s and 1.70 GB for `alpha_qh`, because the HRP ground program is small and the grounding bottleneck that dominates BSP simply does not arise. HRP is where ground-and-solve with lazy heuristics is on the favourable side of the trade-off of Section 5.8, which is the mirror image of the BSP result and worth stating for that reason.

The two guided Alpha configurations also differ in their search behaviour: `alpha_dom` reaches the solution with no backtracking at all, whereas `alpha_qh`

takes 2,142 backtracks for essentially the same number of guesses, with 8.6 times the wall time and 4.1 times the memory use. The two read the same directives; they differ in the evaluation path, native in one case and through Prolog queries over the partial assignment in the other. This is a property of the reference implementation on an aggregate-free heuristic and this thesis does not diagnose it. Its bearing on Section 5.6 is direct, though: on HRP, `la` finds a conflict-free solution and `alpha_dom` a backtrack-free one, whereas `alpha_qh` backtracks. These observations do not establish identical decision sequences or validate the translation against Alpha.

Overall Frontier and Query Engines. Table 5.11 summarizes the maximal instance reach and resource consumption across systems. Regarding query engines, Alpha's Qh mode and the Prolog backend of this thesis both evaluate heuristics by querying an embedded Prolog engine over partial assignments. Consultation costs are of similar magnitude: at BSP $n = 120$, Alpha issues 242 queries at 2.40 ms each, against 120 consultations at 0.45 ms for our Prolog backend. On this family the embedded-engine design is therefore competitive with the reference engine on consultation cost; Section 5.8 shows that the comparison does not hold uniformly, since the cost per consultation depends on how many candidates the rule body enumerates.

Family	System	max solved	Wall (s)	Peak RSS
BSP	`alpha_noheur`	40	30.5	1.2 GB
BSP	clingo `gc_noheur`	150	571.5	15.5 GB
BSP	clingo `la`	160	579.7	19.8 GB
BSP	`alpha_qh`	200	4.2	0.38 GB
BSP	clingo `la_co`	200	0.15	< 0.1 GB
PUP	`alpha_noheur`	—	—	—
PUP	clingo `gc_noheur`	80	124.5	0.7 GB
PUP	clingo `lc`	100	238.2	2.1 GB
PUP	clingo `ga`	200	444.8	24.4 GB
PUP	clingo `la`	200	227.1	12.7 GB
PUP	`alpha_qh`	200	50.9	12.8 GB
HRP	`alpha_noheur`	8	106.0	2.4 GB
HRP	`alpha_qh`	20	15.3	1.70 GB
HRP	`alpha_dom`	20	1.8	0.42 GB
HRP	clingo `la`	20	0.51	< 0.1 GB

Table 5.11: Largest instance solved within the 600 s / 30 GB budget, and its cost, ordered by reach within each family. On PUP, 200 is the last published instance rather than a frontier for the three systems that reach it. Wall time and peak RSS are `runlim` external process measures; Alpha's RSS includes resident JVM memory, not just solver data. clingo rows represent the native C++ backend. A dash marks a configuration that solved no instances.

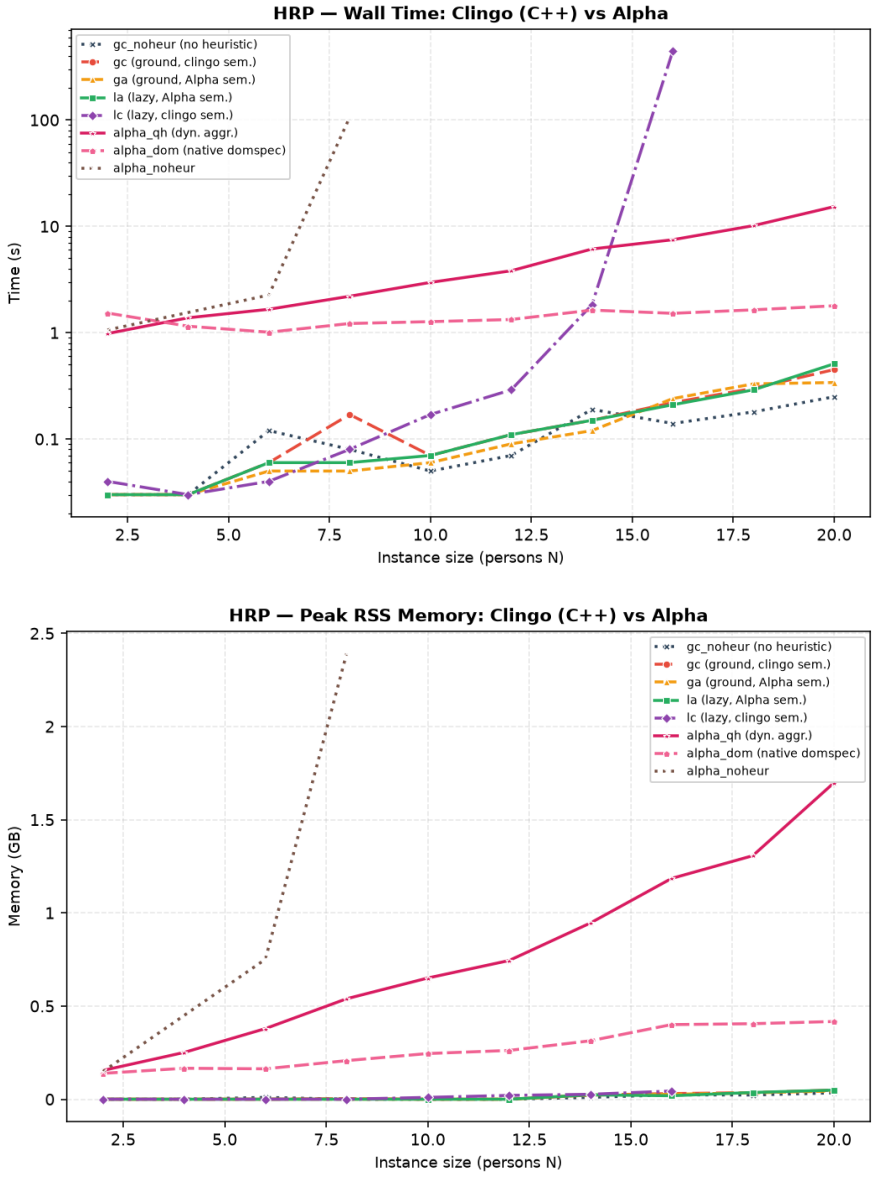


Figure 5.19: HRP benchmark dashboard: the clingo variants against Alpha in all three configurations, including `alpha_dom`. Top: total wall-clock time, on a logarithmic scale to keep the sub-second clingo runs visible next to `alpha_noheur`; bottom: peak RSS. Both are `runlim` external process measures, and Alpha's RSS includes the resident JVM. Only successful runs are plotted, so `lc` ends at $N = 16$ and `alpha_noheur` at $N = 8$.

5.10 The clingo Reading of Partial Information: `gc` and `lc`

The two variants on the clingo side of the semantics axis do not help on any benchmark of this suite, and they fail in ways that are worth separating. This section collects them, because reporting the same negative result three times inside three different family sections obscures both the result and the families.

The outcome should not be read as a defect of clingo's semantics. The heuristics of this suite were written for Alpha, whose reading of `not a` and of an aggregate over choice atoms is the one the encodings assume; `gc` and `lc` evaluate those same bodies under a semantics that requires the information to be settled before it may be read. What the campaign measures is therefore the cost of the mismatch, and the reason the semantics axis of Section 3.1 has to exist as an axis: it is not a free parameter, and picking the wrong value for a given encoding costs everything the heuristic was supposed to buy. The three families exhibit the two possible outcomes of that mismatch.

5.10.1 BSP: inertness

On BSP, `lc` exhibits search statistics that are identical point-by-point to the heuristic-free baseline `gc_noheur`, recording 78,565 choices and 49,993 conflicts at $n = 120$ (Table 5.5 and Figure 5.5). Under clingo semantics, aggregate guards on `c/1` require all elements to be assigned before the sum can be evaluated, and an atom counts as "assigned false" when it is explicitly assigned false in the current partial assignment, by decision or propagation. During search these conditions are never met at a decision point: at $n = 120$ the propagator is consulted on every decision and yields zero applicable candidates across the entire run.

Because `la` and `lc` run on the same native C++ backend and on the same ground problem rules, apart from the semantics annotation, the comparison isolates the effect of the semantics selector for this encoding. In the implemented clingo-like mode, the aggregate binding waits until all its source atoms are assigned. By then, it cannot guide the choices over those same atoms; the negative guard also tests the target's complement.

The ground variant `gc` also incurs the cost of materializing these inactive conditions: the 1,010,200 directives at $n = 100$ enter the propositional formula and multiply the variable count by fifty (Table 5.6). At $n = 120$ this costs 296.6 s of solving against the baseline's 44.8 s, and the frontier drops from $n = 150$ to $n = 130$. A heuristic that never fires is not neutral if the solver has to carry its ground image.

5.10.2 PUP: the same inertness, with the same surcharge

PUP repeats the pattern on a different aggregate. `la` and `lc` produce the same ground problem rules (216,193 rules at $N = 20$), differing exclusively in the semantics selector inside the propagator, so this is again a controlled comparison of the reading alone:

- `la` evaluates dynamic aggregates over currently true atoms, solving $N = 120$ in 48.2 s with only 162 choices.
- `lc` waits for complete assignments, generating zero active candidates during search. At $N = 120$, it makes 49,261 choices and times out.

The unguided `lc` runs also exhibit a non-monotonic failure pattern in Table 5.7 (timing out on `double-80` but solving `double-100`). This non-monotonic outcome shows that instance size alone does not determine search difficulty. With one run per instance, the campaign cannot distinguish instance-specific effects from run-to-run variability or establish the role of restarts.

Here too the ground variant is the worse of the two, and for the same reason: `gc` fails at $N = 80$ with 874,565 choices, *earlier* than the heuristic-free baseline, which solves that same size in 546,549 choices and only fails at $N = 100$. The inert ground directives add variables to the solver and can affect its branching behaviour. The higher choice count is consistent with this effect, although these runs do not isolate changes in VSIDS. On both BSP and PUP, therefore, the standard ground heuristic under the standard semantics is measurably worse than writing no heuristic at all.

5.10.3 HRP: misguidance

On HRP, some directives remain active, but their order no longer matches the intended priority policy. The HRP heuristic has no dynamic aggregates and is organized in five priority levels; under the clingo reading, the negation in the higher-level rules delays them, while the lower-level rules, whose bodies carry no such guard, fire on incomplete information. The heuristic therefore does *not* go quiet: it actively steers, in the wrong order.

The cost is substantial. `lc` requires 66,717 choices at $N = 14$ against 86 for `la`, an increase of nearly eight hundred times; at $N = 16$ it takes 14.2 million choices, five orders of magnitude; from $N = 18$ it no longer finishes (Figure 5.11). Both backends show deteriorating search under `lc`, but Prolog reaches the time limit before exploring a comparable number of decisions on the larger instances.

On the Prolog backend this is one of the configurations that the compiled backend solves and the interpreted one does not: misdirected search and relational query cost compound until `house-14`, which the native backend clears in 1.80 s, exhausts the whole 600 s budget. The Prolog frontier on this family is therefore two sizes short of the native one, $N = 12$ against $N = 16$, and Section 5.8 locates the cost precisely.

5.10.4 What the three cases share

For the lazy clingo-like variant in this suite, the mode of failure reflects the shape of the heuristic body:

- if every guard needs a complete assignment, the heuristic is *inert* (BSP, PUP). Exact agreement with baseline search counters is observed on BSP, but not on PUP;
- if some guards need a complete assignment and others do not, the surviving portion of the heuristic fires out of order and the heuristic becomes *misleading*,

which is far worse than inert (HRP);

- on BSP and PUP, materializing ineffective directives adds grounding cost; this does not extend to HRP, where `gc` solves the entire suite and `lc` does not.

These observations distinguish the applicability of a heuristic from the cost of representing and evaluating it. An inactive lazy heuristic still incurs callback overhead, and inactivity alone does not guarantee the same search trajectory as a separately grounded baseline.

5.11 Two Ground Simulations of Alpha: `ga` versus `ga_weak`

How closely can Alpha semantics be simulated within a ground-and-solve pipeline? Section 3.1.1 introduced two distinct transformations: `ga_weak`, which removes negative atoms from rule bodies, and `ga`, which replaces aggregate equality checks with statically unrolled monotone bounds ($S \leq \#\texttt{sum}\{Y : c(Y)\}$).

Neither transformation is a general-purpose rewriting, and the results below are to be read within the condition established in Section 3.1.1: a sufficient condition for deleting a negative literal while preserving the Alpha reading is that the deleted atom is the target or its complement. BSP satisfies that condition, so the negation transformation is faithful on this family. Only `ga` also reconstructs the dynamic aggregate; `ga_weak` is an ablation, and `ga` remains subject to weight saturation. HRP does not, which is why the corresponding HRP discussion (Section 5.6) treats `ga` as an approximation rather than as a reference.

In BSP, both `ga` and `ga_weak` produce exactly the same $\Theta(n^3)$ ground directives as `gc` (1,120 directives at $n = 10$, 8,440 at $n = 20$, and 27,960 at $n = 30$). Dropping the `sum_value(S)` domain in `ga_weak` does not reduce the grounding size: with the equality binding in place, the grounder must still enumerate all subset sums. Along a search branch, the equality binding becomes available only after all atoms in the sum are assigned, whereas monotone bounds activate incrementally as the partial sum grows. Backtracking can undo either condition.

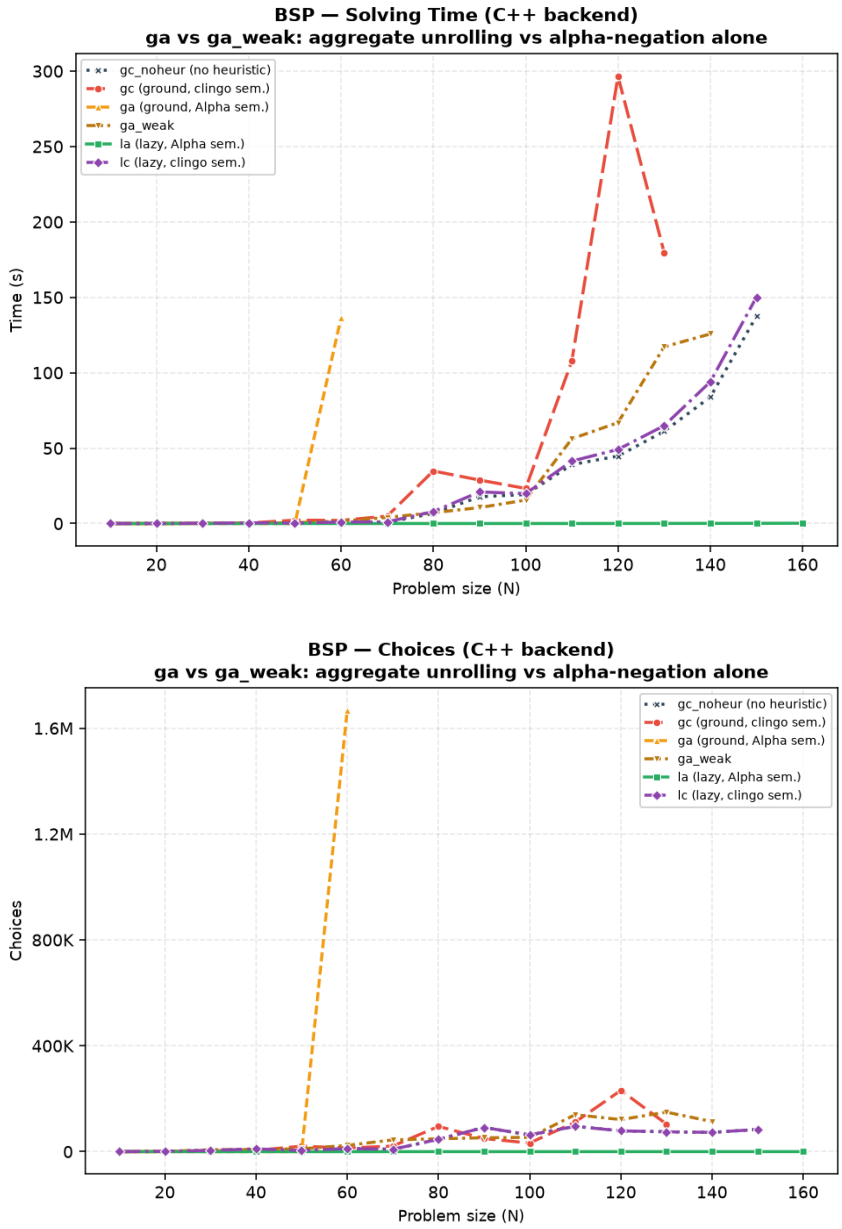


Figure 5.20: Solving time (top) and solver choices (bottom) for BSP comparing the Alpha-like semantics simulations (`ga` and `ga_weak`) against the baseline variants.

Beyond $n = 50$, `ga` deteriorates rapidly as weights exceed the `int16` range and saturate at 32,767: distinct partial sums collapse onto the same weight, and the ranking stops discriminating between candidate decisions (Figure 5.20). Up to $n = 50$ it matches `la` exactly, at n choices and zero conflicts; at $n = 60$ it already requires 1.67 million choices and 136.5 s of solving time, before timing out from $n = 70$ on. The limit is not the cost of the unrolling: at $n = 70$, the first size `ga` fails to solve, that unrolling is 0.6 s of grounding and 36 megabytes over the lazy variant. It is the weight field, into which the reconstructed value of the aggregate has to fit.

`ga_weak` avoids the early search failure of `ga`, but times out from $n = 150$, even though its weight expression is the same as `ga`'s: the equality binding is satisfied only

once every element of the sum is assigned, so the heuristic almost never activates and weight saturation is not the explanation for its lack of guidance. At $n = 120$ its measured search effort is higher and less regular than the heuristic-free baseline, taking about 50% more choices and correspondingly longer to solve (Table 5.5).

Omitting the negative guards without unrolling the aggregate bounds therefore yields no usable guidance: it increases search effort and slightly reduces the solvable horizon. The comparison with `ga` identifies two separate limits on BSP: weight saturation disrupts the simulated ranking, while retaining aggregate equality prevents timely guidance even after the negative guards are removed.

5.12 Evaluating Pre-Materialized Auxiliary Predicates

The `la_aux` variant investigates whether the primary advantage of lazy evaluation stems simply from reducing the number of `__heuristic` directives, or from keeping the evaluation of heuristic conditions entirely outside the ground program. In `la_aux`, heuristic conditions and aggregate values are pre-materialized into auxiliary ASP predicates (`h_b(X,S)` and `h_c(X,S)`), which the lazy propagator then queries during search.

Defining auxiliary rules forces the grounder to instantiate the full Cartesian product of elements and partial sums (X, S). As a consequence, solver variables increase to 132,756 at $n = 50$, compared to just 5,150 in direct `la`.

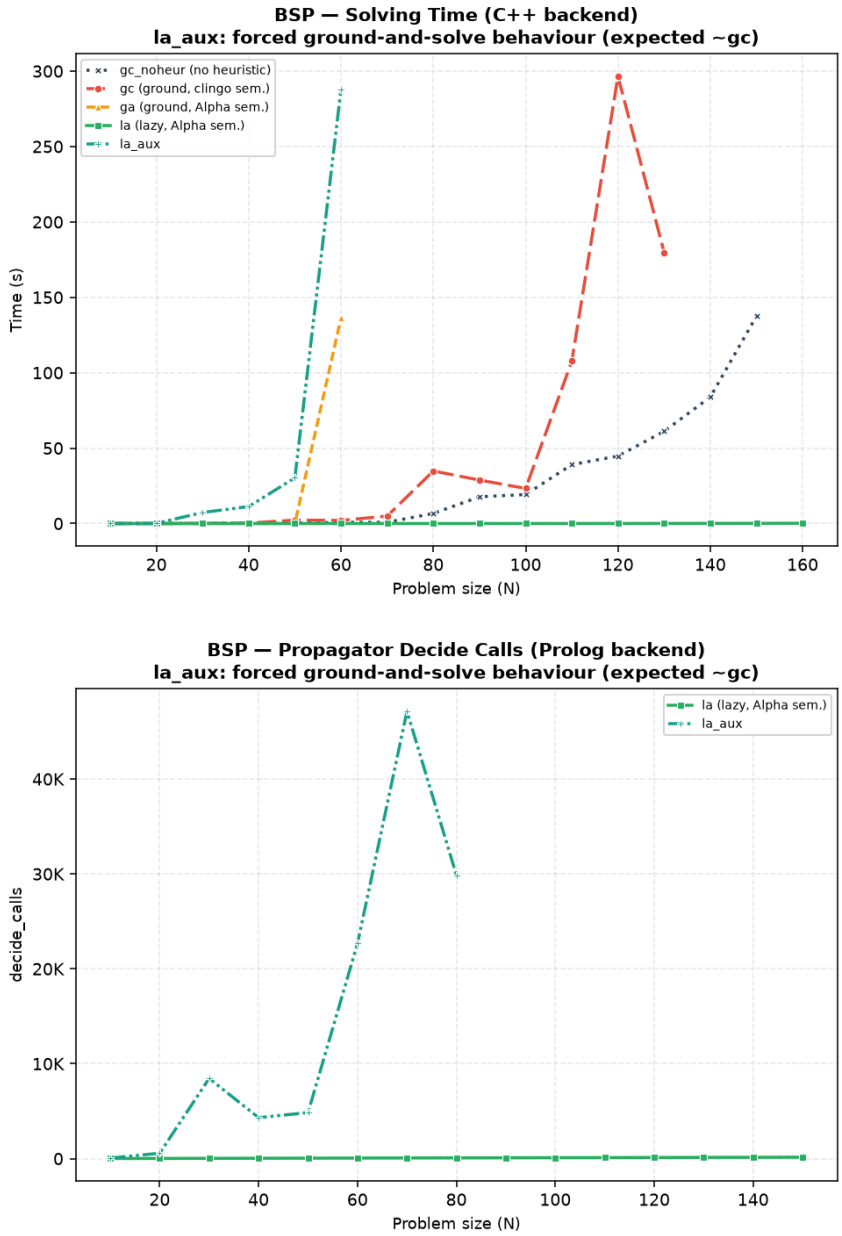


Figure 5.21: Solving time on the C++ backend (top) and number of propagator decide calls on the Prolog backend (bottom) comparing `la_aux` against direct `la` in BSP.

The auxiliary materialization costs performance on both backends (Figure 5.21):

- On C++, maintaining the auxiliary state dominates the cost. At $n = 60$, state synchronization and candidate updates take 284.2 s of the 288.1 s spent solving, while decision callbacks take only 3.4 ms in total. The variant times out from $n = 70$ onward; direct `la` keeps solving time negligible.
- The auxiliary conditions yield no applicable candidates at decision points on either backend. Search therefore needs many more consultations than direct `la`: 573 at $n = 20$ and 8,452 at $n = 30$, instead of n. Prolog benefits most from the cheaper

trail updates: synchronization previously consumed 99.2% of the run at $n = 30$, its last solved size. With foreign predicates, `la_aux` reaches $n = 80$, two sizes beyond the native frontier.

Reducing the number of heuristic directives is therefore not enough on its own: the benefit comes from keeping the evaluation of the heuristic conditions out of the ground program. Once those conditions are materialized as auxiliary rules, the combinatorial cost simply moves from the directives to the base encoding, and the lazy propagator must maintain the resulting auxiliary state during search.

5.13 Effect of the Constraint Formulation

Every result reported for BSP so far shares one confound: the base program is enormous, and it is the same for all variants. The `_co` family removes it. The transformation replaces the quadratic difference check over the two sums with a single constant-bound interval constraint (Section 4.3.1), and it is applied to all six variants of the matrix, so that each has a twin differing from it in the domain encoding alone. Two questions follow, and the family answers them separately: what the reformulation does to the frontier, and, once the base program costs nothing, whether the heuristics still make any difference.

In the standard formulations (`gc`, `la`, and `lc`), this quadratic constraint dominates the grounding phase: at $n = 120$ it produces about 52.8 million propositional rules and requires between two and three minutes of grounding. Direct lazy evaluation (`la`) removes the heuristic directives, but the ground program remains bound by the size of the domain specification.

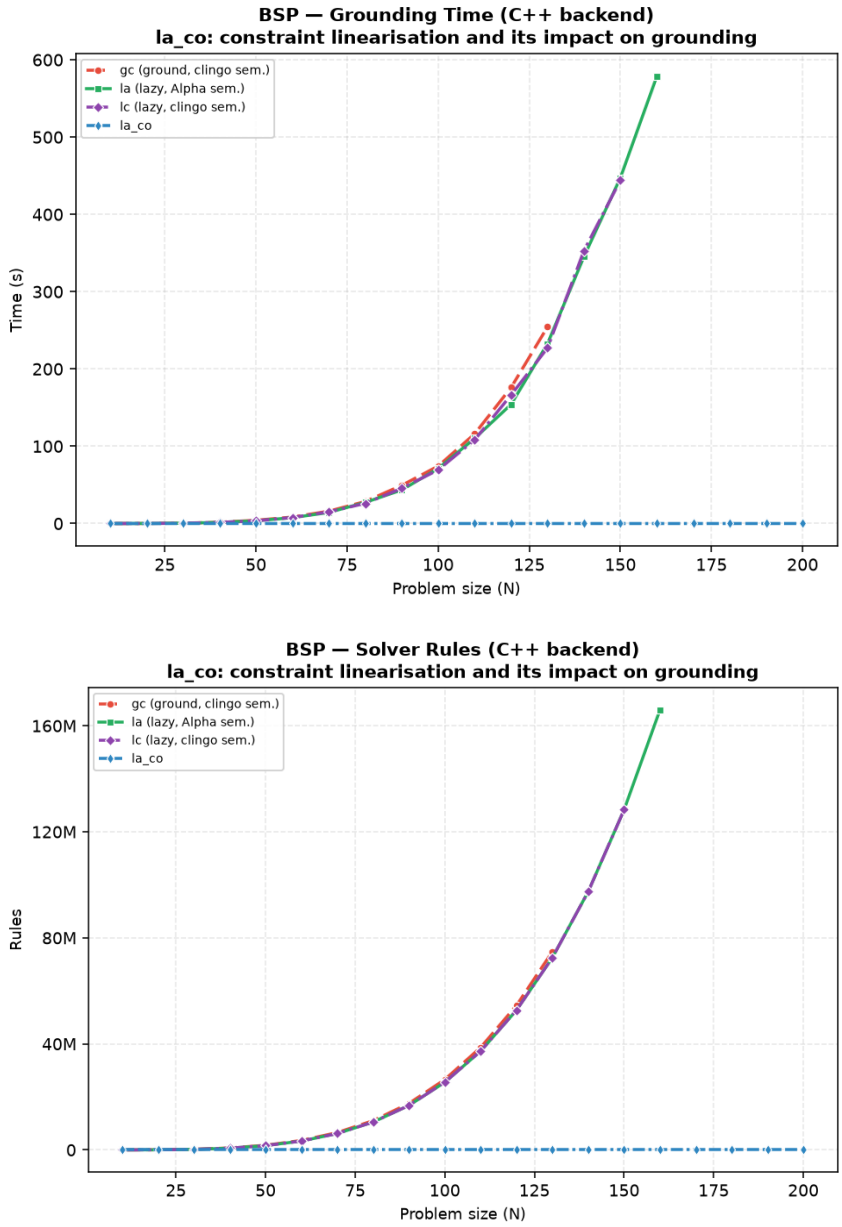


Figure 5.22: Grounding time (top) and propositional rule count (bottom) for BSP on the C++ backend, comparing the compact constraint formulation (`la_co`) against the baseline encodings (`gc`, `la`, and `lc`).

The reformulation changes formula size and grounding time by orders of magnitude (Figure 5.22):

- `gc`, `la`, and `lc` overlap near the top of both plots, sharing the same quadratic base encoding, whereas `la_co` flattens along the horizontal axis. At $n = 120$ the propositional formula drops from 52.8 million rules to 366.
- Grounding all but disappears. Total clingo runtime at $n = 120$ falls from 154.2 s in `la` to 6 ms, while retaining a short conflict-free search (117 choices, compared with 120 in the standard encoding). The same holds up to $n = 200$, which is solved in 32 ms of clingo time; the corresponding `runlim` wall time of 0.15 s reported in

Table 5.11 additionally includes process startup, and the two figures are not in conflict.

The two bottlenecks are therefore independent. Once lazy heuristics remove the search cost, what remains in BSP is dominated by the materialization of the domain constraints, and pairing lazy heuristics with a compact encoding addresses both at once.

5.13.1 The full `_co` family: do the heuristics still matter?

Applying the same reformulation to every variant answers a question the sections above cannot: with the base program reduced to a few hundred rules, does the choice of heuristic still change anything? Table 5.12 reports the whole family at $n = 200$, the largest benchmarked size, which every one of the six variants now solves.

Variant	Rules	Variables	Grounding (s)	Total (s)	Peak RSS	Choices
`gc_noheur_co`	604	201	0.004	0.004	*n/a*	**60**
`gc_co`	8,161,998	8,120,796	44.32	45.35	4,393 MB	100
`ga_co`	8,101,503	40,599	29.99	30.96	1,176 MB	114
`ga_weak_co`	8,161,998	80,796	26.39	27.03	839 MB	107
`la_co`	606	201	0.02	0.03	5 MB	197
`lc_co`	606	201	< 0.01	0.01	*n/a*	**60**

Table 5.12: The `_co` family on BSP at $n = 200$, C++ backend. RSS is marked *n/a* when the resource sampler reports zero on a very short run. All six variants solve every size from 10 to 200, and none spends more than about a second in search at any of them. The standard and compact encodings share their heuristic definitions; Table 5.5 reports both formulations at $n = 120$, whereas this table uses $n = 200$.

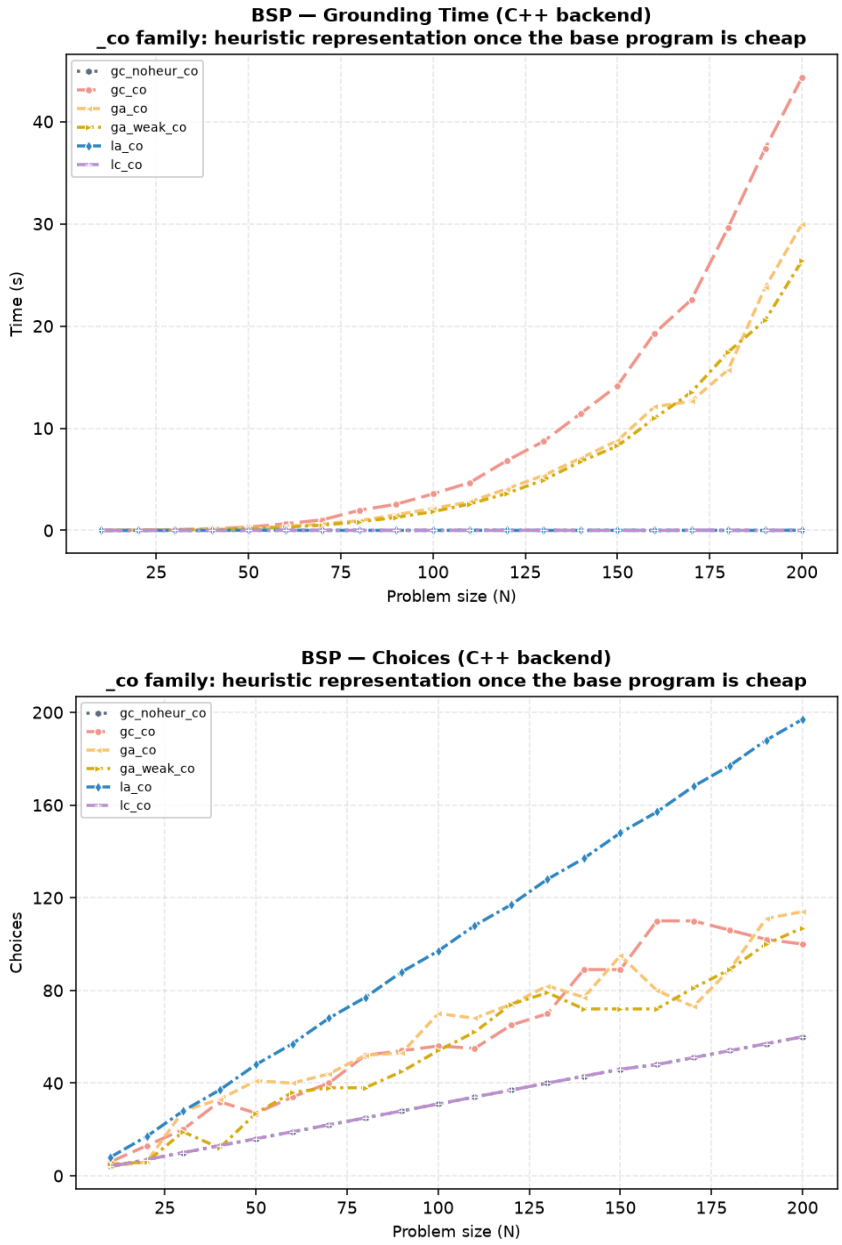


Figure 5.23: The `_co` family on BSP: grounding time (top) and solver choices (bottom). The ground and the lazy variants separate on grounding by three orders of magnitude.

On search, the answer is no. With the compact balance constraint, search becomes inexpensive on the tested BSP instances, and Figure 5.23 shows large differences in grounding time but much smaller differences in choice counts. The heuristic-free `gc_noheur_co` solves $n = 200$ in four milliseconds, taking 60 choices with zero conflicts, because the interval constraint propagates: fixing part of the partition determines the rest without a decision. No variant improves on that. `la_co`, which carries the guidance that on the standard encoding reduced search from 78,565 choices to 120, here takes 197, more than three times the baseline, as its greedy de-

cision order leaves fewer assignments to propagation. The ground variants land in between, at 100 to 114. All variants spend at most about a second in search across this suite. The measured differences therefore offer no practical advantage over the heuristic-free baseline on the compact encoding.

This is worth stating plainly because it bounds the BSP results of Sections 5.3 and 5.10. The dramatic search separations reported there, three orders of magnitude between `la` and the baseline, are separations on a formulation whose difficulty is an artifact of the quadratic constraint. They measure the guidance faithfully, but the thing being guided is a search that a better domain encoding does not have.

On representation, the answer is yes, and it is now measured cleanly. The same table makes the opposite point on the other axis. With the base program at 604 rules, everything above that number *is* the heuristic, and the spread is the widest in the thesis. At $n = 200$, the two lazy variants ground 606 rules in about twenty milliseconds. The very short runs yield low or zero sampled RSS values, which are insufficient to establish an exact memory bound. The three ground variants materialize between 8.10 and 8.16 million rules, spend 26 to 44 seconds doing so, and reach 0.8 to 4.3 GB. That is a factor of 13,500 in formula size and of three orders of magnitude in grounding time. With the base formulation held constant, the comparison isolates the effect of the heuristic representation; previously, the base program contributed tens of millions of rules to every row alike. Section 5.2 counted directives; this measures what they cost once nothing else is in the way.

Two secondary findings. First, `lc_co` reproduces `gc_noheur_co` exactly, the same choice counts and zero conflicts at every size (60 choices at $n = 200$), as `lc` reproduces `gc_noheur` on the standard encoding: the inertness of the clingo reading is a property of the heuristic body, not of the domain encoding it sits in. What the `_co` family adds is the price tag. Its ground twin `gc_co` buys the identical guidance, none, for 44.3 s of grounding, 8.2 million rules and 4.3 GB, and ends up taking 100 choices where the baseline takes 60. The claim of Section 5.10, that an inert *materialized* heuristic is worse than no heuristic, holds here in its sharpest form.

Second, `ga_co` solves the entire range, including the sizes at which `ga` times out. Its weights saturate clasp's 16-bit field exactly as `ga`'s do, since the two encodings differ only in the balance constraint and the weight expression is untouched, but the saturation costs nothing here, because the ranking it corrupts is not what is carrying the run. The limit identified in Section 3.1.4 is therefore a limit on the ranking rather than on the encoding: it bites exactly when the heuristic is doing the work, which on the standard BSP encoding it is and on the compact one it is not.

Chapter 6

Discussion and Conclusions

6.1 Discussion of the Results

The results answer the question that motivated this thesis: clingo can evaluate declarative heuristics during search while retaining its ground-and-solve architecture. The propagator keeps the heuristic description compact and computes its guidance from the current partial assignment. The benefit depends on three factors: the cost of grounding the heuristic, the meaning assigned to its conditions, and the amount of work required to evaluate it at each decision.

6.1.1 What lazy evaluation saves

The clearest saving is in the representation of the heuristic. On BSP, the ground directives grow as $\Theta(n^3)$ and exceed one million at $n = 100$, whereas the lazy description consists of two template facts. This removes the enumeration of possible aggregate values from the heuristic grounding. It does not make the entire solver state constant in size: the base program still has to be grounded, and the propagator maintains runtime data for its targets and aggregates.

The compact BSP encoding makes this distinction particularly clear. At $n = 200$, its heuristic-free baseline solves the instance in four milliseconds with only 60 choices. Guidance is therefore unnecessary on this formulation. Nevertheless, adding ground heuristic directives requires between 26 and 44 seconds of grounding and up to 4.3 GB of peak memory. The lazy variants ground in about twenty milliseconds (Table 5.12). The cost of representing a heuristic can thus remain substantial even when the heuristic provides no search benefit.

With the original BSP encoding, the situation is different. Alpha-like lazy guidance reduces search to n choices with zero conflicts, but grounding the base constraints remains expensive. The largest solved sizes stay close to those of the unguided baseline, and the runs that determine their exact positions approach the time limit. Here, the main result is a reduction in search effort, rather than a robust increase in the largest solvable instance.

PUP shows both the value of guidance and the saving from lazy representation.

The unguided baseline reaches $N = 80$, while the ground Alpha simulation, `ga`, and the lazy Alpha variant, `la`, solve every published `double` instance up to $N = 200$. On the largest instance, the C++ backend reduces total clingo time from 438.6 s to 224.0 s and peak memory from 24.4 GB to 12.7 GB. Both variants reach the end of the available set, so this comparison establishes lower cost at equal measured reach. Larger instances would be needed to determine whether the memory saving also allows `la` to solve instances beyond the reach of `ga`.

6.1.2 Why the semantics matters

Compact representation alone does not ensure useful guidance. The heuristic must also interpret partial information in the way intended by the modeller. This is why the experiments separate representation, ground or lazy, from interpretation, clingo-like or Alpha-like.

The benchmark heuristics were written for the Alpha-like reading. Under that reading, a negative condition can hold while its atom is unassigned, and a dynamic aggregate can be computed from the atoms currently true. The same conditions under the clingo-like reading may become available too late to guide the intended decision.

The consequences differ across the families. On BSP and PUP, the lazy clingo-like variant finds no applicable candidates during search. On HRP, some lower-level directives remain applicable while higher-level ones are delayed, so the heuristic guides decisions in an ineffective order. At $N = 14$, the C++ propagator `lc` run makes 66,717 choices, compared with 86 for `la`. These observations concern the transfer of these particular heuristics between semantics; they do not establish that one semantics is preferable for every encoding.

The ground variants also expose a separate limitation: clasp stores heuristic weights in an `int16` field. Flattening Alpha's global levels into weights preserves their mathematical ordering only when the multiplier is large enough and the resulting values remain representable. On BSP, saturation destroys distinctions needed by `ga`, which loses its greedy behaviour and times out from $n = 70$. The lazy propagator ranks candidates internally and returns a decision literal, avoiding this particular weight-field limit.

This distinction also explains why saturation need not cause a timeout. The compact `ga_co` encoding uses the same weights and solves the full suite because propagation already makes search easy. The impact of a damaged ranking depends on how much the encoding relies on that ranking.

6.1.3 The runtime cost and the choice of backend

Lazy evaluation moves work into search. Its cost depends both on how often the solver requests a decision and on how much work the evaluator performs for each request. Effective guidance can reduce the first factor, but a complex rule body can still make the second expensive.

The two backends illustrate this trade-off. The native C++ backend maintains candidates and aggregates incrementally and has a much lower decision cost on the measured instances. The Prolog backend supports a broader language, but may enumerate many intermediate candidates when evaluating relational bodies. On HRP at $N = 20$, its propagator accounts for approximately 94% of total runtime even though only about one hundred decisions are needed. On PUP and BSP with effective guidance, its share is much smaller (Section 5.8).

Answer-set checks alone do not reveal whether the heuristic provides the intended guidance. Misinterpreted priority annotations and an incorrectly selected Prolog backend can leave the solver producing valid answers while losing the intended guidance. The suite therefore uses flattened weights consistently and checks backend selection and heuristic activity. Decision counters and candidate statistics complement correctness checks by showing whether the mechanism actually participates in search.

6.1.4 What the comparison with Alpha establishes

The external comparison shows where heuristic laziness helps within clingo and where full lazy grounding retains an advantage. On BSP, guided Alpha solves the largest instance with few decisions and no backtracking while avoiding the large base grounding that dominates clingo's runtime. Keeping only the heuristic lazy cannot remove that remaining cost.

On PUP at $N = 200$, the C++ lazy propagator takes 227.1 s of external wall time, compared with 50.9 s for guided Alpha, a factor of about 4.5. Their measured peak process memories are similar, approximately 12.7 and 12.8 GB. Ground `ga` takes 444.8 s and 24.4 GB on the same external measures. Lazy representation therefore reduces the gap substantially, but Alpha remains faster at this size.

HRP gives a different outcome. Its base program is small, and C++ lazy propagator solves the largest instance in 0.51 s, compared with 1.79 s for Alpha's native heuristic mode and 15.33 s for its query-based mode. The two Alpha modes also differ in backtracking, for reasons not diagnosed in this thesis. The comparison therefore supports a conclusion tied to the family and configuration, rather than a general claim that either architecture is faster.

6.2 Limitations

On HRP, the ground Alpha-like variant removes capacity guards and can consequently propose different decisions from the lazy variant. Its results compare the implemented encodings, rather than establish the relative cost of evaluating semantically equivalent heuristics. Preserving the guards' intended effect through clingo priorities is left for future work and requires validation before such an equivalence can be claimed.

The implementation is a research prototype with a defined language scope. The

C++ backend supports a smaller language subset than the Prolog backend, as detailed in Section 3.3.3. The benchmarks use heuristics expressible in both, so their agreement does not establish that every Prolog heuristic can be translated to the C++ propagator language. The propagator is also registered sequentially and has no synchronization layer designed for parallel solving.

The experimental protocol uses one run for each combination of variant, instance and backend. It therefore provides no estimate of run-to-run variance or statistical confidence intervals. Large differences in time are accompanied by structural evidence, such as rule counts and search decisions, but their stability still requires repeated measurements. This is especially relevant to the exact BSP frontier, where the deciding runs finish within 7% of the time limit. The comparison with the earlier Prolog synchronization design likewise spans two campaigns rather than repeated paired measurements.

The benchmark selection places further bounds on the conclusions. PUP contains one instance per size from the `double` family and ends at $N = 200$. HRP remains easy for clasp with the baseline and Alpha-like guidance, so it mainly exposes differences in guidance and evaluation overhead. The original BSP formulation creates a difficulty that largely disappears with a compact balance constraint. Its search results must therefore be interpreted as results for that encoding, rather than evidence that BSP inherently requires a domain heuristic.

On PUP and HRP, the C++ backend uses auxiliary predicates for conditions that Prolog evaluates directly, and candidate ordering can also differ. The backend comparison therefore includes differences in ground representation and search, as detailed in Section 5.7.

Memory measurements are peak RSS for the whole process. They include the embedded Prolog runtime where applicable and the JVM in Alpha, and do not separate grounding memory from search memory. Very short runs can also finish before the sampler records a useful peak. In particular, the close PUP memory figures for Alpha and lazy clingo should be described as similar observations from individual runs.

Finally, parts of the Prolog initialization remain specific to the benchmarks, including a whitelist of static predicates and inference of the constant n from the instance. An auxiliary evaluator that re-grounds programs per decision is retained as a functional reference. General deployment would require a clearer declarative configuration interface and broader validation. The experimental results are limited to the tested encodings and instances; they do not provide an exhaustive validation of all accepted programs.

6.3 Implications

For a modeller, the results suggest examining the domain encoding and the heuristic separately. A compact formulation of the constraints may remove the need for guidance, as on BSP. When a dynamic heuristic is still useful, evaluating it lazily

can avoid paying for all possible aggregate values before search begins. The auxiliary-form experiment reinforces this point: materializing the heuristic body in ordinary ASP predicates can reintroduce the same grounding cost even if the directive itself stays compact.

For solver integration, the main contribution is a working use of clingo's public heuristic interface. Declarative descriptions can be evaluated against the partial assignment and translated directly into branching decisions. The C++ and Prolog implementations offer different balances between runtime cost and expressiveness. Choosing between them requires considering the heuristic bodies and the expected number of consultations, as well as the size of the ground program.

6.4 Future Work

The first experimental priority is to repeat measurements and broaden the instance sets. Repeated runs would help assess the stability of timeout boundaries, while multiple instances per size would separate size effects from instance-specific difficulty. Larger PUP `double` instances would test whether the lower memory use of `la` eventually produces a larger solvable frontier than `ga`. A generator should first be validated against the published instances, and time and memory failures should be recorded separately.

The C++ propagator language could then be extended to support joins between body literals, comparisons between bound values, integer division and modulo, and a systematic treatment of non-numeric symbols. The `doubleV` PUP family from [9] provides a concrete target outside the currently supported subset. Supporting a separate global priority level would also avoid requiring a known bound on intra-level weights for every translation, provided its meaning were made explicit across backends.

An automatic translation from a supported fragment of clingo `#heuristic` directives to the Prolog heuristic rules would reduce manual encoding work. It should make the intended reading of each negative condition explicit and, for the clingo-like aggregate policy used here, insert readiness guards automatically. It could also insert `target_available/1` checks for generated targets. A corresponding translation to the native template representation would expose its supported fragment through familiar ASP syntax; neither translation is implemented in this prototype.

For Prolog, candidate restrictions placed before expensive joins and reuse of query results whose dependencies have not changed are further optimization directions, especially for HRP. Reuse would have to account for both propagation and backtracking, and its benefit remains to be measured.

For Prolog, the next optimization concerns enumeration. Foreign predicates currently materialize a matching relation before yielding its elements. Iterating over indexed data without copying the relation could reduce the cost of bodies with many joins, particularly on HRP. Its benefit would need to be measured across the families, since reducing synchronization already showed that an optimization can shift cost

elsewhere.

Integration could be improved through automatic conversion, weight checks, explicit configuration, statistics reporting, parallel solving, and coordinated tie-breaking. Further benchmarks, including the HRP instances associated with [18], would test transfer to other problems.

6.5 Conclusions

This thesis shows that declarative dynamic heuristics can be evaluated lazily within clingo. A propagator reads a compact heuristic description, evaluates it against the current partial assignment, and supplies the next branching decision. The implementation retains clingo's ground-and-solve pipeline while avoiding the expansion of heuristic aggregate values into large sets of ground directives.

On PUP, lazy evaluation preserves effective guidance while approximately halving total clingo time and peak memory relative to the ground simulation at the largest published instance. On BSP, the compact encoding demonstrates that representation savings remain valuable even when propagation makes the heuristic unnecessary. Together, these results separate the cost of expressing guidance from the benefit of using it.

These savings depend on how the heuristic reads partial information and on the cost of evaluating it during search. The C++ backend achieves low consultation costs through incremental maintenance, while Prolog offers greater expressiveness at a higher and more variable evaluation cost. Neither backend removes the grounding cost of the base problem, which explains the remaining gap with Alpha on BSP and PUP.

The resulting propagator provides dynamic declarative guidance through clingo's heuristic interface. The next steps are to extend the accepted heuristic language and repeat the evaluation on a broader set of instances.

Bibliography

[1] Michael Gelfond and Vladimir Lifschitz. "The Stable Model Semantics for Logic Programming". In: *Proceedings of the Fifth International Conference and Symposium on Logic Programming*. MIT Press, 1988, pp. 1070–1080.

[2] Torsten Schaub. *Answer Set Solving in Practice*. Potassco slide package, University of Potsdam. Version 1.22.0, 4 October 2024. 2024. URL: `https://teaching.potassco.org`.

[3] Martin Gebser et al. *Potassco Guide*. Second edition, version 2.2.0. University of Potsdam. 2019. URL: `https://potassco.org`.

[4] Francesco Calimeri et al. "ASP-Core-2 Input Language Format". In: *Theory and Practice of Logic Programming* 20.2 (2020), pp. 294–309.

[5] Martin Gebser et al. "Domain-Specific Heuristics in Answer Set Programming". In: *Proceedings of the Twenty-Seventh AAAI Conference on Artificial Intelligence*. 2013, pp. 350–356.

[6] Carmine Dodaro et al. "Combining Answer Set Programming and Domain Heuristics for Solving Hard Industrial Problems". In: *Theory and Practice of Logic Programming* 16.5-6 (2016), pp. 653–669.

[7] Antonius Weinzierl. "Blending Lazy-Grounding and CDNL Search for Answer-Set Solving". In: *Logic Programming and Nonmonotonic Reasoning (LPNMR 2017)*. Vol. 10377. Lecture Notes in Computer Science. Springer, 2017, pp. 191–204.

[8] Richard Comploi-Taupe et al. "Domain-Specific Heuristics in Answer Set Programming: A Declarative Non-Monotonic Approach". In: *Journal of Artificial Intelligence Research* 76 (2023), pp. 59–114. DOI: `10.1613/jair.1.14091`. URL: `https://jair.org/index.php/jair/article/view/14091`.

[9] Richard Comploi-Taupe, Gerhard Friedrich, and Tilman Niestroj. "Dynamic Aggregates in Expressive ASP Heuristics for Configuration Problems". In: *Proceedings of the 25th International Workshop on Configuration (ConfWS 2023)*. Vol. 3509. CEUR Workshop Proceedings. CEUR-WS.org, 2023, pp. 75–84. URL: `https://ceur-ws.org/Vol-3509/paper11.pdf`.

[10] Martin Gebser et al. "clasp: A Conflict-Driven Answer Set Solver". In: *Logic Programming and Nonmonotonic Reasoning (LPNMR 2007)*. Vol. 4483. Lecture Notes in Computer Science. Springer, 2007, pp. 260–265.

[11] Matthew W. Moskewicz et al. "Chaff: Engineering an Efficient SAT Solver". In: *Proceedings of the 38th Design Automation Conference*. 2001, pp. 530–535.

[12] Martin Gebser et al. “Theory Solving Made Easy with Clingo 5”. In: *Technical Communications of the 32nd International Conference on Logic Programming (ICLP 2016)*. Vol. 52. OpenAccess Series in Informatics (OASIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016, 2:1–2:15. DOI: 10.4230/OASIcs.ICLP.2016.2.

[13] Jan Wielemaker et al. “SWI-Prolog”. In: *Theory and Practice of Logic Programming* 12.1-2 (2012), pp. 67–96.

[14] Markus Aschinger et al. “Optimization Methods for the Partner Units Problem”. In: *Integration of AI and OR Techniques in Constraint Programming (CPAIOR 2011)*. Vol. 6697. Lecture Notes in Computer Science. Springer, 2011, pp. 4–19.

[15] Gerhard Friedrich et al. “(Re)configuration using Answer Set Programming”. In: *Proceedings of the IJCAI 2011 Workshop on Configuration*. Vol. 755. CEUR Workshop Proceedings. CEUR-WS.org, 2011, pp. 17–24.

[16] Potassco Group. *benchmark-tool: A tool for benchmarking solvers and orchestrating cluster runs*. URL: https://potassco.org/benchmark-tool/ (visited on 09/08/2026).

[17] Armin Biere. *runlim: a tool for sampling time and memory usage of a process*. URL: https://github.com/arminbiere/runlim (visited on 09/08/2026).

[18] Veronika Semmelrock et al. *Learning to Solve and Optimize by Evolving Code*. 2026. arXiv: 2605.31049 [cs.LG]. URL: https://arxiv.org/abs/2605.31049.